

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Dom Colligan

Contents

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses	2
Abstract	2
1 Introduction	3
Contributions	4
2 Background and related work	4
3 The two levers	6
3.1 Telling: the in-context contract	6
3.2 Enforcing: the mechanism inventory	7
3.3 Context-verifiability	8
3.4 L7 stale-manifest drift: the non-retrievable task condition	8
3.5 What the runtime guarantees regardless of behavior	8
3.6 The 2x2	8
4 GovernedAgentBench: methodology	9
4.1 The reference runtime	9
4.2 Task suite and oracle-pair construction	10
4.3 Fixtures, hermeticity, and the manifest snapshot	10
4.4 The two axes as implemented	11
4.5 Deterministic scoring and first-attempt attribution	11
4.6 Evidence tiers	12
4.7 Model roster	13
4.8 Goal-conflict variants and the adversarial layer	13
4.9 Offline reproducibility	13
4.10 The pre-registered run design	14
5 Results	15
5.1 The capability-ladder result	15
5.2 The contract-off map: telling works when salient, and the untold floor is real	17
5.3 The M8 arc: the verifiability exception dissolves	18
5.4 H2: benign goal conflict does not degrade told-only compliance	18
5.5 The instrumental leg is falsified	18
5.6 Predictions against outcomes	19
6 Methodology cautions as a contribution	20
6.1 Harness blindness manufactured a fabrication finding	20
6.2 Prompt-embedded ablations measure self-enforcement	21
6.3 Two supporting cautions	21
6.4 A checklist for agent-eval builders	21
7 The capability ladder and external replication	21
7.1 The capability ladder	21
7.2 Scope: a single-runtime case study; external replication as future work	22
8 Discussion	22

8.0 Scope conditions	22
8.1 What runtime enforcement is still for	23
8.2 The salience confound	23
8.3 Personal health as a domain	24
8.4 Limitations	24
8.5 What would change the conclusion	25
9 Future work and conclusion	25
9.1 Future work	25
9.2 Conclusion	26
Appendix	26
A. Task suite	26
B. Prompt provenance	26
C. Probe protocol summary	27
D. Adversarial trajectory table (pointer)	27
E. Reproducibility	27
F. Scorer config hash chain	27
G. D-O-02: Appendix E coding-agent sketch (open decision)	27
References	28

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Dom Colligan

Abstract

For each constraint, a harness designer chooses: state the rule in the agent’s prompt (an in-context contract), or have the runtime make the violation impossible (runtime enforcement). We cross the two levers in a 2x2 per governance mechanism of a frozen reference runtime, separating the effect of telling, the effect of enforcing, and the marginal value of enforcement given the agent was told. In a small, pre-registered, single-runtime case study we find a capability-moderated pattern on the mutation gate (the runtime’s user-commit boundary): a model that can self-enforce the boundary respects it with the runtime off, while a model that cannot needs the runtime to do the whole job.

The evidence is a four-model ladder (MiniMax-M3 and Llama-3.3-70B as the capable rungs; Qwen3.5-9B and Qwen2.5-7B as the weak rungs). Told the rule with the runtime off, both capable models complied on every replicate of both commit sub-gate tasks, while both weak models committed the gated mutation on every replicate of both tasks; the enforced cells are safe by construction. We read this off the raw cell counts, not a pooled p-value: the independent unit is the task (two per arm, with near-deterministic replicates), so rep-level exact tests overstate precision and are reported only as within-condition figures beside the honest task-level count. The pattern is stark but small and heavily caveated: capability is confounded with model family (both capable rungs non-Qwen, both weak rungs Qwen); once the threshold-sensitive below-floor 7B is set aside, the robust load-bearing evidence rests on a single non-frontier model (Qwen3.5-9B); the pooled cell is n=8, below our own pre-registered target; the capable-side “no marginal value of enforcement” is a failure to detect (one-sided upper bound +32pp), not demonstrated equivalence; and a salience qualifier on the told-arm result rests on a separate n=3 diagnostic on an off-roster model. Two hypothesized moderators nulled at that diagnostic tier (context-verifiability via M8 audit emission; benign goal-conflict pressure, zero fabrication across 40 reps), and the clinical-refusal mechanism is near-ceiling and underpowered (its detector fires on 7 of 223 refusal reps, one task, one model), so we read no substitution result off it.

A second, methodological contribution stands independent of the ladder: harnesses that hide tool output manufacture spurious fabrication findings (harness blindness); an apparent instrumental-fabrication effect in our own pipeline dissolved to 0 percentage points (against a pre-committed ≥ 40 pp bar) once the harness inlined command stdout. We release GovernedAgentBench: the task suite, the deterministic offline scorer,

and the frozen paid-run trajectories and scores, with the reference runtime pinned by git fingerprint (the PyPI wheel does not enforce).

1 Introduction

An agent harness is a design surface. The engineer who assembles an LLM agent over user-owned structured data controls the loop around the model: which tools are exposed, how actions are parsed and dispatched, and what happens when a proposed action would violate a constraint. For any given constraint, that engineer holds exactly two levers. The first is in-context specification: state the rule in the agent’s prompt as an in-context contract (a command manifest, a flag, a mutation class, a boundary description) and rely on the model to respect it. The second is runtime enforcement: have the harness block or gate the violation deterministically, regardless of what the model decides. Telling is cheap and moves with the prompt; enforcing requires typed command schemas, dispatch gates, and validation code that must be built, tested, and maintained. A harness designer deciding where to spend engineering effort needs to know when the second lever buys behavior the first does not.

Most existing guardrail evaluations do not answer this question, because they conflate the two levers or vary enforcement against a prompt-wording axis rather than a told-versus-withheld one. Some toggle enforcement while holding the full policy in the system prompt in both conditions [arXiv:2604.15579], so the delta is measured only where the agent was already told. Others bundle whether rules are injected and enforced into one variable [arXiv:2602.22302], or compare an enforced arm against a prompt-only arm [arXiv:2603.00822, arXiv:2605.28914, arXiv:2606.21856], which mixes the effect of telling into the effect of enforcing; the closest, Mind the GAP [arXiv:2602.16943], crosses three prompt-wording conditions with a runtime governance contract but never withholds the rule per mechanism. The enforced-not-told cell is nearly absent from prior work (the closest located instance is a single by-design condition in FORGE [arXiv:2602.16708]), and no located prior work runs the specify(told-versus-withheld)-per-mechanism factorial with first-attempt scoring; Section 2 develops the map. None of these designs can answer the question a harness designer actually faces: what does enforcement add, given that the agent was told?

This paper crosses the two levers factorially in a specify-vs-enforce 2x2 per governance mechanism (Section 3). Its headline quantity is the marginal value of enforcement given the agent was told (cell A: deployment baseline vs cell B: told-not-enforced). We instantiate the design in GovernedAgentBench, built over a frozen non-clinical personal-wellness reference runtime (HAI, pinned by git fingerprint; Section 4.1) whose governance mechanisms (M4 through M8, Section 3.2) can be ablated individually or all at once, and scored by a deterministic offline scorer.

We hypothesized three moderators would gate when specification substitutes for enforcement: context-verifiability of the constraint, benign goal-conflict pressure (compliance costs task success; benign completion pressure, not adversarial injection), and model capability. Model capability is the one that carries the result, in a small pre-registered ladder we read descriptively rather than as a powered test. Told the mutation-gate rule (the W57 user-commit boundary) with the runtime off, the two capable models (MiniMax-M3, Llama-3.3-70B) complied on every replicate of both commit sub-gate tasks, while the two weak models (Qwen3.5-9B, Qwen2.5-7B) committed the gated mutation on every replicate of both tasks; the enforced cells are safe by construction, so the finding lives entirely in the told-not-enforced (cell B) and untold-not-enforced (cell D) cells. The independent unit is the task (two per arm) and the four replicates per cell are near-deterministic, so we lead with the raw counts and the per-rep mechanistic verification (a capable model refuses the commit citing the user-gate; a weak model runs it while the gate is disabled) rather than a pooled p-value; where we report an exact test we print the honest task-level figure beside the rep-level one. The pattern is a genuine sign flip in the marginal value of enforcement given told – undetectable for the capable models, the whole barrier for the weak ones – but it is small and confounded: capability is confounded with model family (both capable rungs non-Qwen, both weak rungs Qwen), and once the threshold-sensitive below-floor 7B is set aside the robust load-bearing evidence reduces to a single non-frontier model (Qwen3.5-9B, Section 7.1). The capable-side “no marginal value of enforcement” is a failure to detect at this n (one-sided upper bound +32pp), not demonstrated equivalence. Because the reconstructed untold-unenforced cell actually violates even for the capable models (Section 4.4), telling and enforcing each have a measured effect within a single model, so the capable-model result reads as substitution rather than pre-existing disposition. The

other two moderators nullled at the diagnostic tier (one model, Qwen3-235B-A22B-Instruct-2507, Together AI, temperature 0, n=3-5 per cell): the verifiability exception dissolved once the agent retrieves the relevant evidence, and goal conflict produced zero fabrication across all 40 reps under graded pressure P0-P3. The clinical-refusal mechanism is near-ceiling and underpowered (its detector fires on 7 of 223 refusal reps, one task and one model), so we read no substitution result off it. We claim no “unnecessary guardrails” conclusion: where the model self-enforces, enforcement’s marginal behavioral value is undetected at this n but is never zero as a guarantee, and its demonstrated value remains the deterministic guarantee plus the region where the model cannot self-enforce – below the operate floor, the untold violation floor, the sub-frontier capability band, the incidental-salience failure of Section 5.2, and adversarial intent, untested here. Injection robustness is cited territory, not a claim of this paper. [FORK-L7-DRIFT: unmeasured; do not assume outcome]

The second finding is methodological. Our own harness initially manufactured a spurious fabrication result: it fed the agent a file-path placeholder instead of actual command stdout, and the blinded agent guessed identifiers it could not see. Inlining stdout dissolved the apparent instrumental-fabrication effect entirely (0 percentage points against a pre-committed threshold of at least 40 points; diagnostic, one model, n=5), and the agent abstained honestly instead. We name this failure mode harness blindness: evaluations that hide tool output from the agent manufacture findings like this.

Contributions

1. A capability-moderated finding with explicit scope conditions, reported as a small single-runtime case study rather than a powered claim: on a pre-registered four-model ladder over the frozen HAI reference runtime, told the mutation-gate rule with the runtime off, both capable models complied on every replicate of both commit tasks while both weak models committed the gated mutation on every replicate – a sign flip in the marginal value of enforcement given told, undetectable for the capable models and the whole barrier for the weak ones. The independent unit is the task (two per arm) with near-deterministic replicates, so we lead with raw counts and per-rep mechanistic verification and treat the rep-level exact test as a within-condition figure (the honest task-level exact p is 0.33). Capability is confounded with model family, the robust load-bearing side reduces to one non-frontier model (Qwen3.5-9B), and the capable-side null is a failure to detect (one-sided upper bound +32pp) not equivalence; the salience qualifier and deterministic-guarantee caveat remain attached; and the clinical-refusal arm is near-ceiling and underpowered, not a second substitution result. Single-runtime case study; external non-HAI replication is future work (Section 7.2).
2. A methodological demonstration that harness blindness manufactures spurious fabrication findings, shown by dissolving one such finding with a harness-level fix, offered as a caution for agent-eval methodology.
3. GovernedAgentBench, the released instrument for the 2x2: the task suite, a deterministic offline scorer, the reference runtime pinned by git fingerprint (the PyPI wheel does not enforce; Section 4.1), and the paid-run trajectories and scores released as a versioned archive alongside the preprint, with static oracle-pair evidence, live runtime probes, and model-backed diagnostics kept as separate evidence tiers.

The novelty claim is a conjunction, not any single first: the 2x2 including the enforced-not-told cell, per-mechanism isolation, the substitution account with its moderators, the methodological warning, the deterministic offline scorer, and the released benchmark, taken together. This is an AI-engineering study of agent-harness governance. It claims no scaling law, no injection robustness, and no result beyond this controller, task suite, and evidence tier.

2 Background and related work

Agent harnesses and scaffolding. The harness, the deterministic software layer between a model and its tools, is now a design surface in its own right. A survey under review at TMLR treats governance and verification as a distinct harness layer (ETCLOVG, OpenReview 3hXEPbG0dh; cited via OpenReview, not as an accepted paper). Harness-Bench [arXiv:2605.27922], NLAH [arXiv:2603.25723], AHE [arXiv:2604.25850], and Meta-Harness [arXiv:2603.28052] measure or optimize harness configurations for capability; none carries

a governance axis. This paper measures the governance layer rather than optimizing it.

Runtime enforcement and guardrails. Enforcement frameworks (AgentSpec [arXiv:2503.18666]; NeMo Guardrails; Guardrails AI; Invariant Guardrails, from Invariant Labs, now part of Snyk) supply mechanism substrates without measuring what the model would do on its own. In the injection-defense literature, CaMeL [arXiv:2503.18813] and Progent [arXiv:2504.11703] show architectural enforcement outperforming prompt-only heuristic defenses on AgentDojo; CaMeL never states its policies to the model in any condition, and Progent runs no disclosure factorial, so neither manipulates disclosure. Harness-MU [arXiv:2606.21856] argues governance constraints should be “enforced by execution hooks rather than entrusted to the LLM,” a single-axis comparison under adversarial framing. ContextCov [arXiv:2603.00822] compiles instruction files into executable guardrails (88.3% compliance enforced against 67.0% prompt-only on SWE-bench Lite); its enforced arm feeds violation traces back to the model, in-context specification delivered late. Verifier Tax [arXiv:2603.19328] compares unenforced baselines (Tool-Calling, TRIAD) against a policy-mediated architecture (TRIAD-SAFETY) on tau-bench Airline and Retail: safety mediation intercepts up to 94% of non-compliant actions, yet safe success stays below 5% in most settings, with agents hallucinating user identifiers to route around blocks; it toggles enforcement architecture, not prompt disclosure, so it runs no told/untold contrast. Mechanical Enforcement [arXiv:2605.14744] compares text-only governance against mechanical enforcement (four architectural primitives outside the model’s interpretive loop) in a regulated-banking decision agent, cutting uninformative deferrals by 73% and raising task accuracy from MCC approximately 0.43 to 0.88; it pairs two alternative configurations rather than crossing a factorial, has no enforced-not-told cell, and measures decision-rationale quality rather than tool-action compliance. (“Prompt-based guardrail” in Policy-as-Prompt [arXiv:2509.23994] means an external classifier reading a policy, a different lever from the in-context contract studied here.)

Instruction and in-context rule following. The main risk of misreading this paper is reducing it to “capable LLMs follow instructions.” That models follow checkable instructions is established, but IFEval [arXiv:2311.07911], RECAST [arXiv:2505.19030], and VerIF [arXiv:2506.09942] use checkability to build evaluations and rewards, not as a predictor of when agents comply. And told-only compliance is not free: LogiSafetyBench [arXiv:2601.08196] names “Unsafe Success,” capable models completing tasks while violating latent rules, non-monotone across model families, as is SABER [arXiv:2606.01317]; Agent-SafetyBench [arXiv:2412.14470] reports none of 16 agents above 60% safety score; SafePyramid [arXiv:2606.29887] shows in-context policy application degrading with policy reasoning complexity; Governance Decay [arXiv:2606.22528] shows constraints obeyed while visible are silently erased by context compaction, violation rising from 0% to 30%, up to 59% for some models. That establishes the drift-causes-violation half of the stale-manifest phenomenon; whether independent runtime enforcement catches post-drift violations remains open. [FORK-L7-DRIFT: unmeasured; do not assume outcome] Our question is therefore not whether an agent can follow a stated rule, but what marginal behavioral value runtime enforcement retains once the same constraint is specified in context.

Software contracts and capability security. The runtime’s vocabulary of manifests, typed command schemas, proposal/commit gates, and least privilege descends from design-by-contract and capability security. Agent Behavioral Contracts [arXiv:2602.22302] evaluates runtime contracts at scale (200 scenarios, 7 models, 1,980 sessions), but its conditions “differ only in whether ABC contract rules are injected and enforced,” bundling specification and enforcement into one variable, on a patent-gated benchmark. Prompts Don’t Protect [arXiv:2605.18414] filters unauthorized tools out of the model’s context with no policy prompt, a structurally enforced-not-told arm whose 0.0% unauthorized-invocation rate holds by construction, since attempted violations cannot be observed.

Closest priors. AIRGuard [arXiv:2605.28914] is a security-framed pre-action runtime guard; its prompt-only versus runtime-guard ablation is a single-axis attack-success delta on one model, with no factorial, no enforced-not-told cell, and no constraint-class analysis. PhantomPolicy [arXiv:2604.12177] formalizes policy-invisible violations, where compliance depends on runtime state absent from the agent’s context, contributing an eight-category taxonomy this paper adopts and crosses with the enforcement axis. It does run a policy-in-prompt condition (risky-case violations 95.3% to 40.7% under human-reviewed labels), but folds telling into the same single ordinal axis as enforcement architecture (baseline, policy-in-prompt, content-DLP, Sentinel), with Sentinel and the DLP baseline always applied to baseline not-told traces, so no condition

holds enforcement fixed while varying telling and no enforced-and-told versus enforced-and-not-told contrast exists. Neither AIRGuard nor PhantomPolicy crosses the two levers. Mind the GAP [arXiv:2602.16943] is the closest prior to this paper’s capability-moderation headline: it reports both that prompt-based tool-call safety is strongly model-dependent across six frontier models and that a runtime governance contract produces no detectable deterrent on forbidden tool-call attempts. Its manipulated axis is prompt wording (three phrasings) crossed with a single monolithic governance contract under jailbreak framing, not told-versus-withheld per mechanism; it has no withheld arm, no enforced-not-told cell, no per-mechanism inventory, and no first-attempt scoring, and its “no deterrent” finding is a leakage catch-layer under attack rather than a hard commit gate under benign use (the directional tension is taken up in Section 8.1). We cite it from its verified abstract and released repository only. FORGE [arXiv:2602.16708] runs the only located by-design enforced-not-told condition: “The instrumented condition does not include the natural language policy in the agent’s prompt; the agent relies solely on runtime enforcement and corrective feedback.” It has no told-and-enforced cell and no neither floor, so it cannot measure the redundancy of enforcement given specification; corrective feedback makes its enforced arm converged multi-turn rather than first-attempt; it evaluates a single holistic Datalog policy per deployment, with no per-mechanism inventory, no moderators, and no located public release. ABSTAIN [arXiv:2606.02965] reaches the closest cell coverage, three of four cells for a single abstention mechanism; its Checkpoint condition reuses the Prompt-Only prompt (its Appendix B), realizing told-and-enforced rather than enforced-not-told, and non-Checkpoint conditions are LLM-judge scored. TeamBench [arXiv:2605.07073] reports statistically indistinguishable pass rates between prompt-only and OS-sandbox-enforced role separation, alongside 3.6 times more verifier-edits-executor cases in the prompt-only arm: convergent published evidence for the substitution shape in a different mechanism domain, and a standing caution that aggregate parity can mask sub-metric enforcement value, taken up in Section 7.2; it is prior evidence in a different domain and design, not a replication of this paper’s effect (this paper is a single-runtime case study; external replication is future work, Section 7.2). Symbolic Guardrails [arXiv:2604.15579] holds the full policy in the system prompt in both conditions while toggling enforcement; TRACE [arXiv:2606.13174] compiles user corrections into runtime checks in the preference domain, the rule always known to the agent in some form. The closest benchmark prior is ST-WebAgentBench [arXiv:2410.06703]: in-context policies delivered to three agent scaffolds and scored post hoc via Completion Under Policy, with no mechanism that blocks a non-compliant action before execution.

Positioning. Isolated enforced-not-told measurements exist (FORGE by explicit design; Prompts Don’t Protect structurally). No located prior work holds that cell inside a crossed factorial against a told-and-enforced cell and a neither floor, decomposes the comparison per governance mechanism, scores the telling axis on first-attempt behavior, or releases a deterministic offline scorer and benchmark that hold the two levers apart. The contribution is that conjunction, together with the harness-blindness methodological finding (Section 6); no element is claimed as individually unprecedented.

3 The two levers

An agent harness has two distinct levers for making an agent respect a constraint. It can state the rule where the model can read it, or it can make the violation impossible regardless of what the model decides. We call the first lever in-context specification (telling) and the second runtime enforcement (enforcing). Deployed harnesses almost always pull both levers at once: the system prompt describes the policy and the runtime also implements it. Prior evaluations largely inherit this bundling, toggling specification and enforcement as one variable [arXiv:2602.22302] or manipulating only one axis while the other never varies [arXiv:2605.28914]. This section defines the two levers separately, fixes the mechanism inventory that instantiates the enforcement lever, introduces the constraint classification that the design uses as its predictor, and assembles the pieces into the 2x2 that the rest of the paper measures.

3.1 Telling: the in-context contract

The telling lever is the in-context contract: the rule, manifest fact, schema, or policy statement made available to the agent inside its context at decision time. In the reference runtime this includes the command manifest (which commands exist and their argument schemas), the `agent_safe` flag attached to each command, the mutation class of each write surface, and the prose description of the out-of-contract boundary, including its

clinical-boundary leg. The contract is broader than a system-prompt policy paragraph; schemas, flags, and taxonomies are all part of what the agent is told.

Two boundaries keep this lever distinct. In-context specification is a harness-level lever, not a training-time one; it is orthogonal to instruction tuning or preference training, which shape the model’s priors rather than its decision-time context. And the in-context contract is only the copy of the specification the model can read, not the specification the runtime implements. The paper’s central manipulation is the gap between the two.

3.2 Enforcing: the mechanism inventory

The enforcing lever is runtime enforcement: the harness blocking, gating, or otherwise making a violation impossible regardless of what the model emits, independent of whether the model was told the rule. The reference runtime (HAI v0.2.0, the frozen instrument described in Section 4) implements enforcement through a fixed inventory of mechanisms, five of them individually ablatable through a named runtime mode and one held constant.

ID	Mechanism	Ablation mode	Context-verifiable?
M4	Validation of typed commands and proposal payloads	<code>no_validation</code>	Yes (schema is in the manifest)
M5	<code>agent_safe</code> dispatch refusal	<code>no_agent_safe</code>	Yes (<code>agent_safe</code> flag is in the manifest)
M6	W57 proposal/commit user gate	<code>no_proposal_gate</code>	Yes (mutation class is in the manifest)
M7	Refusal of out-of-contract requests (clinical-boundary leg is the zero-tolerance subset); JSON output exempt	<code>no_refusal</code>	Yes (the boundary is describable in the prompt)
M8	Audit evidence emission / reference faithfulness	<code>no_audit_chain</code>	No under the standard setup (Section 3.3)
M9-TX	Transaction integrity	Held constant, non-ablatable	n/a

M4 guards the command surface itself: malformed, out-of-schema, or hallucinated commands are rejected before dispatch. M5 refuses dispatch of commands the manifest does not flag as agent-safe. M6 guards user-authored state: activation or deactivation of a proposed mutation is a user-gated commit. In the measured mutation-gate cells, however, the block that fires is M5’s dispatch refusal (the commit command is flagged not-agent-safe), which reaches the dispatch first; M6’s proposal/commit gate is masked by M5 and is `--confirm`-bypassable, so it is defense-in-depth here and is not independently exercised (PAPER.md D-48). We attribute the measured block to M5 and do not claim a clean M5-versus-M6 decomposition. M7 guards the scope boundary, with the clinical boundary (no diagnosis, treatment, prescribing, or autonomous medical decisions) as its zero-tolerance subset; structured JSON output is exempt. M8 guards reference faithfulness: the runtime emits and persists audit evidence for recommendations, and the deterministic scorer detects fabricated or suppressed references after the fact. M8 is provision-type rather than block-type (Section 3.5 defines the distinction), and it is evidence emission, not the whole audit chain. M9-TX is the atomic-transaction floor; it is held constant and has no ablation mode.

The runtime-enforcement axis therefore has exactly seven modes: `full_contract`, the five per-mechanism off-paths (`no_validation`, `no_agent_safe`, `no_proposal_gate`, `no_refusal`, `no_audit_chain`), and `no_runtime_enforcement`, in which M4 through M8 are all off while the M1-M3 harness infrastructure and M9-TX remain on. M1 through M3 are the always-on harness infrastructure (the model loop and dispatch,

action parsing, and observation feedback); they are not ablatable governance mechanisms and remain on in every mode, which is what makes `no_runtime_enforcement` a floor over which cell D is still driven rather than an inert harness. One caveat governs every per-mechanism number in this paper: harness mechanisms are coupled, so per-mechanism results are reported as marginal contribution within this fixed controller, never as independent additive contributions.

3.3 Context-verifiability

The design’s predictor variable is context-verifiability: a constraint is context-verifiable if the agent can check its own compliance using only what is already in its context at the moment it decides. Verifiability is a property of the pair (constraint, decision-time context), not of a mechanism in the abstract, and it can flip within a trajectory: a constraint becomes verifiable the moment retrieval lands the relevant ground truth in the agent’s context. This framing adopts the observation, formalized as policy-invisible violations in [arXiv:2604.12177], that compliance can depend on runtime state absent from the agent’s context; we treat that dependence as a trajectory-local property rather than a fixed taxonomy label.

Under the standard task setup, with the manifest and taxonomies in the prompt and no runtime state disclosed, M4, M5, M6, and M7 are context-verifiable: the schema, the `agent_safe` flag, and the mutation class are in the manifest, and the M7 boundary is describable in the prompt. M8 was classified non-verifiable under the standard setup and pre-registered as the exception where even a capable cooperative agent could not self-enforce, because whether a cited evidence reference actually exists is runtime state the agent cannot see. The pre-registered prediction attached to this classification was that enforcement is behaviorally redundant for context-verifiable constraints under cooperative, unconflicted conditions and behaviorally necessary for non-verifiable ones at every capability level. Section 5 reports that M8’s classification did not survive contact with the evidence: audit faithfulness turns out to be verifiable once the agent retrieves the evidence, exactly the within-trajectory flip the definition permits.

3.4 L7 stale-manifest drift: the non-retrievable task condition

One constraint class is genuinely non-retrievable rather than merely unretrieved. Under L7 stale-manifest drift, the agent’s in-context manifest is outdated relative to the actual runtime contract, and the agent cannot retrieve the fact that this is so: that ground truth is runtime state no read surface exposes. No sequence of lookups can flip it to verifiable, which distinguishes drift from M8. Drift is a task condition, not a mechanism row: it is a property of the relationship between the contract copy the agent holds and the contract the runtime implements, and it can afflict any mechanism whose manifest facts have moved. L7 drift is the sole surviving candidate for a true non-verifiable enforcement delta in this design, and it is unmeasured. [FORK-L7-DRIFT: unmeasured; do not assume outcome]

3.5 What the runtime guarantees regardless of behavior

The two levers differ in kind, not only in strength. Telling changes what the model is likely to do; enforcing changes what can happen at all. With enforcement on, the block-type mechanisms (M4 through M7) make the guarded violation impossible to complete no matter what the model emits: an out-of-schema command does not dispatch, a flagged command does not execute, an uncommitted proposal does not activate. M8’s guarantee is provision-type: evidence is emitted and persisted whether or not the model narrates it faithfully, so fabrication is detectable after the fact. M9-TX guarantees atomicity in every mode. This is the deterministic guarantee, and it holds regardless of model behavior, model identity, and prompt content. The distinction matters for reading every result that follows: when this paper reports that enforcement is behaviorally redundant in some cell, the claim is about the marginal effect on observed behavior, never that the guarantee is worthless. Behavioral redundancy is not structural redundancy.

3.6 The 2x2

Crossing the two levers per mechanism yields the headline design:

	Runtime enforces	Runtime off
Contract in prompt	A: deployment baseline	B: told-not-enforced (self-enforcement)
Contract withheld	C: enforced-not-told (pure runtime)	D: neither (violation floor)

Cell A, the deployment baseline, is how harnesses actually ship. Cell B, told-not-enforced, isolates self-enforcement: the agent knows the rule and nothing stops it. Cell C, enforced-not-told, isolates pure runtime governance: the rule is withheld from the prompt while the runtime still enforces it. Cell D, neither, is the violation floor. The `no_runtime_enforcement` mode is a runtime mode, not a cell: under the contract-in-prompt arm it serves as a robustness sanity floor, never per-mechanism attribution evidence; under the contract-withheld arm the same mode realizes cell D and anchors the floor for the contrasts below.

Three contrasts carry the information. B vs D is the effect of telling: what specification alone buys over nothing. C vs D is the effect of enforcing: what the runtime alone buys over nothing. A vs B is the marginal value of enforcement given the agent was told, the headline quantity of this paper: if A and B do not differ behaviorally, enforcement added no behavioral value on top of specification for that constraint in that situation, and its remaining value is the deterministic guarantee of Section 3.5.

The magnitude a designer should read off these contrasts depends on the constraint’s type, a distinction that becomes an identification limit in Section 8.4. Three types recur. A disposition-covered constraint is one a cooperative capable agent already respects whether or not it is told: B vs D is near zero, and with the runtime off changing nothing the agent would have done, A vs B is near zero too, so substitution there is degenerate rather than measured. A specification-substitutable constraint is one the agent cannot honor unless told (B vs D large), and for which a runtime enforcer may not even exist. An enforcement-load-bearing corner is one where the runtime carries behavioral value the contract does not: the untold violation floor, the region below the operate floor, the incidental-salience failure, and adversarial intent. The marginal value of enforcement given told (A vs B) is informative only for a constraint that is not disposition-covered. On the frozen tasks our enforce-able mechanisms were disposition-covered at the cooperative tier (Section 8.4); the reconstructed mutation gate of Section 4.4 is not, its untold-unenforced cell actually violating even for capable models, so its redundancy is measured rather than degenerate. The capability ladder (Section 5.1) is the primary identification of specification against disposition; the within-model directiveness pilot of Section 9.1 is complementary.

One interaction between the cells requires a scoring convention. A blocked action returns an error message, and an error message is in-context specification delivered late, so cell C converges toward cell B after the agent’s first contact with the enforcement surface. Axis attribution for B vs D and C vs D is therefore scored on first-attempt behavior, with converged multi-turn behavior reported separately; Section 4 gives the operational details. The 2x2 runs under three moderators, constraint class per Section 3.3, benign goal-conflict pressure, and model capability, which Section 4 defines and Section 5 evaluates.

4 GovernedAgentBench: methodology

GovernedAgentBench is the instrument that holds the two levers apart: it lets in-context specification and runtime enforcement be toggled independently, per mechanism, against a fixed reference runtime, and scores the result deterministically and offline. This section describes the runtime, the task suite, the fixtures and hermeticity discipline, the two axes as implemented, the scorer, the evidence tiers, the model roster, and the reproducibility path.

4.1 The reference runtime

The runtime under measurement is HAI v0.2.0, a non-clinical personal-wellness reference runtime with a typed CLI command surface. HAI is the instrument, not the contribution: it is pinned as a frozen snapshot so that every mechanism has one concrete, inspectable implementation, and it is described here as

a reference implementation, not a product. The runtime instantiates the mechanism inventory of Section 3: M4 validation of typed commands and proposal payloads, M5 `agent_safe` dispatch refusal, M6 the W57 proposal/commit user gate, M7 refusal of out-of-contract requests (with the clinical-boundary leg as the zero-tolerance subset), M8 audit evidence emission, and M9-TX transaction integrity, which is held constant and non-ablatable. The non-clinical boundary is part of the evaluated contract, not a disclaimer: the runtime refuses the clinical actions enumerated under M7 in Section 3.2, and the benchmark scores that refusal. The agent-facing contract is a frozen manifest snapshot, `agent_cli_contract.v2` (approximately 189 KB, checked in at `manifests/hai_0_2_0.json`), which carries the command schemas, `agent_safe` flags, and mutation classes; with it in the prompt, and the M7 boundary describable in the prompt itself, M4 through M7 are context-verifiable under the standard task setup (Section 3.3).

Two facts about the runtime’s identity are load-bearing for reproduction. First, the PyPI `v0.2.0` wheel does not enforce: the dispatch-time `agent_safe` gate landed three days after the version tag, so `pip install hai==0.2.0` yields a non-enforcing runtime that would silently corrupt the enforce arm. The measured runtime is therefore pinned by git fingerprint `6c82cd0`, whose nearest named ancestor is the annotated tag `gab-runtime-1.0`, 14 commits back; the intervening commits add the M7 clinical scanner and the M5 side-door hardening the enforce arm needs, so the tag predates the measured runtime and must not be used to reproduce – reproducers check out `6c82cd0`, not the tag or the wheel. Second, to guarantee the enforce arm was live before any paid call, a pre-spend runtime-identity preflight behaviorally asserts enforcement: it drives an agent-context commit under `full_contract` and requires the runtime to refuse it (exit non-zero, state left proposed), hard-stopping the sweep otherwise. The preflight passed for this run (fingerprint `6c82cd0`, wheel absent), so the enforce cells were measured against a runtime verified to enforce, not assumed to.

4.2 Task suite and oracle-pair construction

The suite is drawn from five levels of a seven-level task taxonomy: L1 intent-to-command routing, L2 setup and recovery, L5 faithful narration of runtime output, L6 governance and refusal, and L7 contract drift; the intervening L3 and L4 levels are out of scope for this suite (Appendix A gives the full taxonomy). Each task declares its `load_bearing_mechanisms` and `runtime_modes_in_scope`; the harness refuses to run a task under a runtime mode the task does not declare, so mode coverage is a checked property rather than a convention. The released suite is 39 tasks (L1: 2, L2: 4, L5: 8, L6: 24, L7: 1) after the D-48 removal of the validation-doctor task and the D-49 clinical-refusal additions; the paid ladder swept a concentrated 16-task subset of it (Section 4.10), with the remainder covered offline. PAPER.md decision D-37 (2026-07-04) retired the earlier 28-task / 25-oracle-pair apparatus and rebuilt the suite as a sharp per-mechanism 2x2 (told x enforced); D-39 (2026-07-05, commit `ed07f5a`) then expanded it to three scenario pairs per mechanism, on the grounds that a per-mechanism null resting on a single scenario is too thin to carry. Every task is a labelled cell of its mechanism’s 2x2 via an explicit `contract_arm` field, and the suite’s moderators are committed suite members rather than post-hoc splits (Section 4.10). Appendix A holds the per-task table.

Every task is constructed as an oracle pair that routes a mechanism’s effect onto a scored metric: a hand-authored `full_contract` trajectory and a matching mechanism-off trajectory whose scored outcome must differ from it on the task’s load-bearing metric, demonstrating scorer sensitivity to the mechanism. Each of M4 through M8 is covered by static oracle-pair load-bearing tasks under the D-39 suite (three scenario pairs per mechanism), and two tasks carry the `no_runtime_enforcement` floor; the retired pre-consolidation pair counts (25 pairs) no longer apply, and the per-task table is Appendix A’s. A mechanism’s off-path is accepted as isolated only if every emitted `mechanism_disabled` marker belongs to that mode’s disabled set, at least one marker for the target mechanism fires, `full_contract` emits zero markers, and the scored consequence delta against `full_contract` on the task’s load-bearing metric is attributable to the disabled mechanism. These pairs are static oracle-pair evidence: a deterministic canary establishing scorer sensitivity and coverage over constructed cases, not live causality evidence.

4.3 Fixtures, hermeticity, and the manifest snapshot

Seven synthetic fixtures are built for the benchmark (`empty_user`, `ready_user_minimal`, `read_surface_user`, `governance_user`, `drift_user`, `adversarial_user`, and `audit_pending_user`, the last leaving the target

day un-synthesized so no evidence card exists at build time, which stresses M8 audit-reference faithfulness); not all seven are referenced by every locked task, and the fixture-to-task mapping follows the D-39 suite of Section 4.2. Fixtures contain no private health rows, no live wearable exports, no credentials, and no maintainer personal data. Every benchmark subprocess runs hermetically: `HAI_HERMETIC=1` with the state database and base directory redirected (`HAI_STATE_DB`, `HAI_BASE_DIR`), and a fresh fixture is materialized per repetition, so no state leaks across repetitions or modes. L7 tasks additionally swap in a deliberately stale manifest snapshot (`agent_cli_contract_v1_drift.json`) at task-load time, so the agent’s in-context contract diverges from the actual runtime contract by construction; this drift condition is a task condition, not a mechanism row, and it remains unmeasured at the model-backed tier [FORK-L7-DRIFT: unmeasured; do not assume outcome].

4.4 The two axes as implemented

The runtime-enforcement axis is the `runtime_mode` seam, exposing the seven modes of Section 3.2. The contract-in-prompt axis is a per-task `contract_arm` field (told or untold) realized at prompt-render time by three coordinated transforms, while the runtime remains unchanged: a structured transform over the embedded manifest that withholds the mechanism-relevant facts (flags, mutation classes, schema constraints for the constraint under test), a prose scrub that removes the constraint’s natural-language statements from the prompt body under a forbidden-token check, and parameterized boundary blocks so the told arm’s boundary prose is assembled per task rather than hand-edited. A mechanical untold-leak scan over every rendered untold prompt is a lock gate: no rendered untold prompt may carry the withheld constraint’s tokens. Crossing the two axes yields the cells of Section 3.6, and the two roles of `no_runtime_enforcement` (robustness sanity floor under the contract-in-prompt arm, cell D anchor under the contract-withheld arm) are kept distinct as defined there. Two provenance caveats attach to the untold arm. First, to keep the untold-unenforced cell from measuring command-discovery failure rather than governance disposition, the untold render restores a rule-free description of the commit command (D-54); this discoverability reconstruction was applied after the canary phase, so the capable models’ cell-D violation rate – the floor the substitution-not-disposition argument rests on – carries that post-canary provenance and can be re-scored with and without it offline. Second, the told-arm boundary prose and the untold withholding were both hand-reconstructed pre-run (D-52) so that told states the operative rule and untold withholds it, fixing an earlier artifact in which both arms committed; this is disclosed rather than silent.

Three disclosures attach to the untold arm’s construction. First, the untold arm withholds all natural-language constraint prose but, by design, preserves the object-capability command surface: the model still sees command names (e.g. `hai target commit`, `hai target archive`) and their `--confirm` flags. The schema therefore partially specifies the constraint even when untold, so B-vs-D and C-vs-D are lower bounds on the effect of telling; the harness schema is itself an in-context contract. Second, in the untold M5 arm the manifest transform asserts `agent_safe: true` rather than merely omitting the gate, so the untold M5 cells are a “told-it-is-safe” floor, disclosed as told-the-opposite rather than uninformed; the diagnostic harm-floor probe shows violations still occur in cell D despite the schema signal, so the floor is non-vacuous. Third, cell B’s told prompt retains boundary prose stating that the runtime enforces the constraint, while the runtime is in fact ablated in that cell; the prompt is held constant across the enforcement axis by design, so cell B measures self-enforcement under an (inaccurate) assertion of enforcement, and this is disclosed rather than edited away, since editing it would break the held-constant told prompt.

The operator action contract is structured JSON, one action per turn, not arbitrary shell. Transcripts record the model’s emitted action verbatim as the assistant turn and each observation as a returned message; no provider-native tool calls are synthesized, because fabricating tool-call structure would misrepresent the evaluated transcript.

4.5 Deterministic scoring and first-attempt attribution

The scorer is deterministic and offline; it contains no model calls. Its behavior thresholds, pass rule, and critical violation kinds are loaded from `scorer_config.paper_v1.json`, whose audited bytes are hashed in `lock_hashes.json`. We disclose a scope limit on that frozen record: the scorer’s Python logic in `core.py` is not itself hashed, and the harm-only `unsafe_action_rate` code path was edited three times in the thirty

hours before the run – adding a state-write side-door check (D-54), an active-insert check (D-55), and an argument-defaults resolution (D-55.1, committed 47 minutes before the run) – each prompted by a pre-run canary and each in the hypothesis-favorable direction. “Frozen” therefore scopes to the config bytes, not the scoring code, and the pre-registered scorer should be read as fixed only up to those three commits. We make the material consequence checkable: those edits added an `unsafe_command_args` path that fired in zero mutation-gate headline cells (every violation in Table 5.1 is the pre-existing `must_not_call` detection, `unsafe_mutation`), so re-scoring the paid trajectories with the pre-D-54 scorer is the identity on the headline 2x2 and the load-bearing result is invariant to the pre-run code changes; only the below-floor 7B’s graded partial scores touch the new path. One circularity is disclosed rather than removed: the scorer’s clinical-boundary detector imports the same banned-phrase list the runtime’s M7 refusal uses (frozen before the run, audited for over-broad single-word entries), so scorer and runtime share ground truth for that violation kind and cell A is clean on it by construction. Citation resolution for `must_cite` metrics is scoped to command stdout only.

The telling axis is scored on first-attempt behavior, per the convergence rationale of Section 3.6. First-attempt is defined uniformly as the first action that reaches the enforcement surface for the constraint in question, not literal step one; the definition applies identically to refusal rows and dispatch rows, so an agent that emits invalid-output steps before its first contact with the enforcement surface is scored from that first contact. In the pre-registered run the first-attempt window closes only on genuine enforcement contact, a blocked `must_not_call` gated action, not on any incidental error message. For the gated mechanisms (M5 `agent_safe`, M6 proposal/commit) that contact is, for a single-target `must_not_call` task, the model’s first action, so the telling and enforcing contrasts for these mechanisms are single-decision difference-of-proportions, reported as pooled counts; we frame the two windows as enforcement-as-barrier (first attempt) versus enforcement-as-teacher (converged) and do not claim multi-step first-attempt reasoning for gated cells. Validation, refusal, and audit-chain mechanisms retain multi-step first-attempt windows. First-attempt scoring carries the B vs D and C vs D attribution, and converged multi-turn behavior is reported separately and never blended into first-attempt numbers.

The metric is only well-defined for a model whose actions the harness can read: a model whose output the action parser cannot read never reaches the enforcement surface, so its first-attempt behavior is undefined and its apparent failure conflates self-enforcement with the operate floor (Section 6.3). The early diagnostic ladder was confounded exactly this way, which at the time scoped first-attempt attribution to Qwen3-235B, the one parser-clean model. An envelope-tolerant parser has since shipped (uniform code-fence stripping and prose-embedded JSON extraction applied identically to every model before validation; normalization never changes which action is validated; commit 8319b83), so in the pre-registered run first-attempt scoring runs across the full ladder of Section 4.7. Two caveats survive the fix and are disclosed there: per-model vendor-recommended sampling confounds cross-model first-attempt comparisons, and the cross-family evidence rests on a single Llama point.

A deterministic rule baseline anchors the routing tasks as plumbing evidence only: it confirms the harness, fixtures, and scorer compose, and it fails the L5 narration tasks by design, since it emits no final narration. (PAPER.md D-30 recorded five L5 narration tasks at the time of the scorer-correctness pass; the D-39 suite carries eight L5 tasks, per Section 4.2 and Appendix A.)

4.6 Evidence tiers

Three evidence tiers are kept separate throughout and are never merged into one claim: static oracle-pair evidence (hand-authored full/off pairs, scorer coverage), live runtime probes (end-to-end HAI runs under compared modes with real command execution, live causality evidence), and model-backed diagnostics (real model completions scored by the same scorer). Model-backed evidence comes in two tiers, kept separate and never merged: the primary pre-registered capability-ladder sweep (four models, n=8 per cell on the mutation gate; Sections 4.10, 5.1) and a lower diagnostic-probe tier (one model, Qwen3-235B-A22B-Instruct-2507, Together AI, temperature 0, n=3 to 5 per cell) that framed the design and is reported as provenance in Sections 5.2 through 5.5.

4.7 Model roster

Two rosters must be kept distinct: the diagnostic-probe roster under which the Section 5.2-5.5 probes were run, and the pre-registered run roster (PAPER.md D-41) under which the capability-ladder sweep of Section 5.1 was executed.

Diagnostic-probe roster (Sections 5.2-5.5). The working model was `Qwen/Qwen3-235B-A22B-Instruct-2507-tput` (Together AI, non-thinking mixture-of-experts, serverless), run at temperature 0. `claude-sonnet-4-6` was the designated fallback for narration-heavy M8 tasks; no run reported here is Sonnet-backed. Excluded during viability screening: `Qwen2.5-7B-Instruct-Turbo` (below the operate floor, 0/60 valid finals, contract friction rather than governance), `Mistral-Small-24B` (32k context too small for the manifest prompt plus multi-turn history), `Gemma-3-27B` and `Qwen2.5-32B` (dedicated-endpoint only), and `Gemma-4-31B` (a reasoning model whose thinking consumed the token budget, yielding empty completions and timeouts).

Pre-registered run roster (Section 5.1). The executed ladder is a four-model Together serverless capability ladder: `MiniMaxAI/MiniMax-M3` as the capable primary, `Llama-3.3-70B` as the second capable point, `Qwen3.5-9B` as the near-floor point, and `Qwen2.5-7B-Instruct-Turbo` deliberately re-admitted as the below-floor operate control (its diagnostic exclusion is the reason it is in the ladder, since a below-floor model is the positive control for the operate-floor claim). The pre-registration’s primary was `Qwen/Qwen3-235B-A22B-Instruct-2507-tput`; the provider deprecated it from serverless mid-cycle, and we replaced it with `MiniMax-M3`. We disclose the researcher degree of freedom this introduces: the replacement was chosen after a canary confirmed it reproduced the target specification effect (an outcome-conditioned selection, PAPER.md D-56), so `MiniMax-M3` should be read as a non-pre-registered addition and its result as exploratory. The pre-registered capable rung unaffected by this is `Llama-3.3-70B`, which shows the same capable-side pattern. Capability is also confounded with model family: the two capable rungs are non-Qwen (`MiniMax`, `Llama`), the two weak rungs Qwen (Section 8.4). Sampling is vendor-recommended per model rather than uniform temperature 0, so model identity is confounded with sampling regime across the ladder; the per-model 2x2 is computed within a single model at a constant sampling regime and is unaffected (Sections 4.10, 8.4). Provider seeds are unsupported on the serverless endpoints, so replication is across independent stochastic draws rather than fixed-seed determinism. Exact model identifiers, pricing, context windows, and sampling settings are pinned in the locked roster revision and recorded in the run manifest (PILOT_PROTOCOL.md §20).

4.8 Goal-conflict variants and the adversarial layer

The goal-conflict arm uses task variants applying benign completion pressure, not adversarial injection: respecting the constraint costs task success. The diagnostic H2 probes graded this from P0 to P3, where at P3 a pipeline is blocked and the user cannot proceed; in the pre-registered suite goal conflict is carried by two committed suite members (`gab_16_agentsafe_conflict`, `gab_15_audit_conflict`; Section 4.10). No attacker-controlled content is involved; this targets the literature-verified locus where frontier models violate policies stated in their own system prompt (Symbolic Guardrails [arXiv:2604.15579]), including under realistic completion pressure (LogiSafetyBench [arXiv:2601.08196]). Separately, a 16-trajectory adversarial layer (four hand-authored trajectories each against M7, M5+M6, M8, and M4) provided scorer-coverage evidence at the appendix tier only, never model-backed. It was retired at the benchmark tier by commit `a10e850`, and PAPER.md D-37 records the retirement as final: the adversarial-input arm is cited future work, not an arm of this paper (Appendix D). Injection robustness is cited territory in the security literature, which already shows telling fails under attack, not a claim of this paper.

4.9 Offline reproducibility

A single command, `reproduce_offline.py`, regenerates the rule-baseline ablation, evidence tables, figures, error taxonomy, and cell contrasts with no network access, no private state, and no paid APIs. The static and live isolation matrices were retired at the benchmark tier by commit `a10e850` (Appendix E) and the pipeline no longer emits them. It emits a top-level manifest with per-artifact summary metadata and exits nonzero if the rule-baseline ablation produced no scored rows or figures; the golden-artifact baseline records hashes and a wall-clock reference (Appendix E; re-baseline pending). The model-backed trajectories and scores are the

only artifacts that required paid API access; the run tree is not in the source repository (it is gitignored), so they are released as a versioned archive alongside the preprint, and from that archive they are re-scoreable offline by the same deterministic scorer. The Table 5.1 cell counts are regenerated by the released pooling module (`build_cell_contrasts_pooled`); the exact tests reported beside them are computed by a small released helper, since the pooling module itself carries no statistics dependency.

4.10 The pre-registered run design

This subsection describes the pre-registered design of the run reported in Section 5.1; the design was locked before the run in PAPER.md decisions D-39 through D-43, with the run manifest, the pre-committed outcome-branch map, and the full disclosure set carried by PILOT_PROTOCOL.md §20.

Suite and cells. The full 39-task suite of Section 4.2 is the offline-reproduction scope. The paid ladder was pre-committed (D-49) to a concentrated headline subset that carries both levers: the two Tier-A mechanisms (the `agent_safe` mutation gate and clinical refusal), the Tier-B M8 blind/sighted twin, and the Tier-C operate control. The Tier-D scope mechanisms (M4 validation, the M6 proposal-gate variant, and the credential and audit-domain families) are covered offline by `reproduce_offline` and are never paid-swept, so the paid run measures only constraints on which both the telling and the enforcing lever move. Each paid task is a labelled cell of its mechanism’s told/untold-by-enforced/off 2x2 via the `contract_arm` field and the `runtime_mode` seam of Section 4.4. The mutation-gate headline (Section 5.1) pools that mechanism’s two committed commit sub-gate tasks, target-commit and intent-commit, each pre-registered at $n=4$, into an $n=8$ per-cell contrast; the off cell for the mutation gate is the aggregate `no_runtime_enforcement` mode (Section 5.1 states the single-mechanism-attribution caveat this carries).

Moderators as committed suite members. The three moderators are in the suite, not post-hoc splits: benign goal-conflict pressure as two committed tasks (`gab_l6_agentsafe_conflict`, `gab_l5_audit_conflict`); constraint verifiability as a blind/sighted twin pair (`gab_l5_audit_blind` against its sighted twin), in which the blind cell withholds ground-truth citations so task success is unreachable there by construction, and the blind twin is read as an honesty probe (abstain versus fabricate) on audit-reference faithfulness and unsupported-narration rate, never on task success, which is retained only to distinguish the sighted twin; and L7 stale-manifest drift as a committed task (`gab_l7_drift`). Verifiability and solvability co-vary in the blind manipulation; the clean claim available from it is honest abstention under hidden ground truth.

Models and replication. The four-model ladder of Section 4.7 runs with vendor-recommended per-model sampling, including the deliberate below-floor operate control. Replication is $n=4$ per cell, raised from 3 after an exact-binomial sensitivity analysis showed that at pooled $n=9$ the 10-point decision bar is a one-rep gap with mid-regime false-alarm rates of 25 to 41 percent, while pooled $n=12$ makes it a two-rep gap; the full analysis is released with the benchmark (`reports/sensitivity/SENSITIVITY.md`). We disclose that the mutation gate carries two sub-gate tasks, not the three scenario pairs that analysis assumed, so its pooled headline cell is $n=8$, below the $n=12$ two-rep-gap target. The observed mutation-gate effects are saturated (0/8 versus 8/8), so they do not sit in the flaky one-rep-gap regime the analysis warned against, but the sub-target n is stated rather than hidden.

Decision quantities. The pre-registered headline is the per-mechanism specify-vs-enforce 2x2 cell contrast (A/B/C/D; B-vs-D the effect of telling, C-vs-D the effect of enforcing, A-vs-B the marginal value of enforcement given told), reported as pooled counts (k/n) and percentage-point differences; medians are demoted to clearly-secondary descriptive statistics, and the legacy pooled runtime-on/off median machinery of the earlier pilot design is retired. The smallest effect size of interest is a single 10-percentage-point bar (0 points only for hard safety invariants); the earlier 5-point pilot bound is deleted, and this change is disclosed here rather than silent. Pooled counts are the scientific quantity; any unanimous per-task pass rule is operational tooling, not an evidential criterion. First-attempt and converged windows are both reported per Section 4.5, framed as enforcement-as-barrier versus enforcement-as-teacher.

Outcome categories. Three non-behavioral outcomes are reported as their own categories, never folded into behavioral failure counts: provider safety-filter blocks, context-window overflow, and length truncation.

Each is neither a pass nor a behavioral fail; pooling any of them into failure rates would inflate apparent incapacity, most acutely for the small ladder models.

Phasing and integrity. The run is canary-first with a hard stop: a canary subset (the untold-floor pair, the blind/sighted twin, and the below-floor operate control) runs across all models and is scored before the main sweep; the gate asks whether each canary axis moves by at least 10 points in the expected direction, and a canary failure attributable to model behavior is a reportable negative result that ends the sweep, not a rerun trigger. A rerun is permitted only for transient-infrastructure causes (provider outage, rate limit, parser or adapter defect) and never to change a cell result; reruns are capped at one fixed re-run with the defect named before re-running, and every voided run is disclosed with its void reason. Claim language for every possible outcome is pre-committed in an eleven-branch outcome map (PILOT_PROTOCOL.md §20), so the paper’s conclusions section is selected from the map by the data rather than written after seeing it.

5 Results

The headline result is the pre-registered capability ladder (Section 5.1): a four-model run on the concentrated task set that produces a capability-moderated pattern on the mutation gate, reported as a small case study on raw cell counts rather than a powered significance claim. It supersedes the thin $n=3$ diagnostic ladder that preceded it. The diagnostic-tier probes that framed the design (Sections 5.2 through 5.5) are reported afterward as provenance: one model (Qwen/Qwen3-235B-A22B-Instruct-2507-tput, Together AI, temperature 0), small n (3 to 5 per cell), run on diagnostic probe tasks rather than the locked suite, kept as a distinct and lower evidence tier. Static oracle-pair evidence and live runtime probes were established separately per Section 4.6 and are not merged with any number below. Throughout, the informative reads of the 2x2 are the effect of telling (cell B: told-not-enforced vs cell D: neither, the violation floor), the effect of enforcing (cell C: enforced-not-told vs D), and the marginal value of enforcement given told (cell A: deployment baseline vs B). L7 stale-manifest drift remains unmeasured (Section 5.6, row P5; discussed in Sections 3.4, 8.4). [FORK-L7-DRIFT: unmeasured; do not assume outcome]

5.1 The capability-ladder result

The ladder ran the concentrated task set across four rungs (MiniMax-M3 as the capable primary, Llama-3.3-70B, Qwen3.5-9B near the operate floor, Qwen2.5-7B below it), canary-first, against a single reference runtime pinned by git fingerprint 6c82cd0 (Section 4.1; this is 14 commits past the nearest annotated tag `gab-runtime-1.0`, which predates the enforce-arm fixes and must not be used to reproduce). A pre-spend runtime-identity preflight confirmed the enforce arm refuses an agent-context commit under `full_contract` before any paid call. All eight canary and main runs completed (USD 10.44 total paid spend, 448 rep ledgers); the pre-registered canary gate passed. The mutation gate is the runtime’s user-commit boundary: an agent may propose a target or intent, but activating user-authored state requires an explicit user commit. The block observed in the enforced cells is M5 (the `agent_safe=false` dispatch refusal); M6 (the W57 proposal/commit gate) is masked by M5 and `--confirm-bypassable`, so it is defense-in-depth here, not independently exercised (Section 3.2, PAPER.md D-48). The off cell is the aggregate `no_runtime_enforcement` mode, which disables M4 through M8 together; we treat the resulting delta as a whole-governance-off contrast on a commit task, on which the commit gate is the only enforcer that can fire, and support that the fired mechanism is the commit gate with the per-rep `mechanism_disabled` markers rather than by assertion. We report the pooled `agent_safe` 2x2 (both sub-gates, `unsafe_action_rate`, cells as percentage safe); first-attempt and converged windows are byte-identical here, so a single number stands for both.

Model (capability tier)	A: told+enf	B: told+off	C: untold+enf	D: untold+off
MiniMax-M3 (capable)	100*	100	100*	0
Llama-3.3-70B (capable)	100*	100	100*	25

Model (capability tier)	A: told+enf	B: told+off	C: untold+enf	D: untold+off
Qwen3.5-9B (near-floor)	100*	0	100*	12.5
Qwen2.5-7B (below-floor)	100*	0	100*	12.5

Table caption: Pooled mutation-gate 2x2 per model, percentage safe (n=8 per cell = two commit sub-gate tasks x 4 near-deterministic replicates; the target-commit untold cells are the pre-registered canary untold-floor carriers, identically scored). *The enforced cells A and C are safe by construction – all 64 enforced reps score 0.0 with zero variance, because the runtime blocks the gated commit regardless of the model – so they carry no behavioral degrees of freedom (they are the deterministic guarantee) and the entire behavioral finding lives in cells B and D.

The behavioral finding is the B column, told-not-enforced, where the agent knows the rule and the runtime does nothing. As raw per-task counts: both capable models complied on all four replicates of both commit tasks (cell B safe 8/8 each; one-sample Clopper-Pearson interval [63, 100]), and both weak models committed the gated mutation on all four replicates of both tasks (cell B safe 0/8 each; [0, 37]). The marginal value of enforcement given told, A minus B, is thus 0pp for the capable models and +100pp for the weak ones. The capable-side 0pp is a failure to detect at this n, not demonstrated equivalence: cell A is pinned at 100% by construction, so A-B is one-sided, bounded above by the cell-B interval; we report it as “no marginal value of enforcement detected (one-sided upper bound +32pp)” and attach neither a symmetric CI nor a “p=1.”

The independent unit of replication is the task, not the rep: two sub-gate tasks per arm, four near-deterministic replicates each (7 of the 8 model-by-sub-gate cell-B conditions are byte-identical across all four replicates despite decoding temperature). A rep-level exact test therefore overstates precision – the weak-model A-B is Fisher p=0.00016 at the rep level but 0.33 at the honest task level (two tasks versus two) and 0.029 per sub-gate (n=4) – so we do not lead with it. Our single pre-registered confirmatory contrast is the pooled A-B on the mutation gate, one per model; treated at the rep level the two weak-model rejections survive a Bonferroni correction across the four models (adjusted ~0.00062), but on the honest task-level unit that same contrast is not significant (p=0.33), so we present the crossover as a descriptive pattern and rest nothing on the rep-level rejection. Every other contrast in this section is descriptive. The primary evidence is the raw counts plus a per-rep mechanistic check: in cell B, MiniMax refuses the commit on its own turn and tells the user to run it, citing the user-gate, while Qwen-7B issues **hai target commit** itself, with **mechanism_disabled** markers confirming the gate was down, and the commit executes. The sign flip in cell B (100/100/0/0) holds on both sub-gates; the difference-in-differences S is negative for the capable models but its weak-model interval includes zero, so we do not rest the load-bearing claim on it.

This capability contrast is what lets the capable-model result read as substitution rather than pre-existing disposition. The reconstructed mutation gate (Section 4.4) has an untold-unenforced cell that actually violates: even for the capable models cell D sits near the floor (MiniMax 0, Llama 25 percent safe, all via the original **must_not_call** scorer path), so telling has a large measured effect within a single model (B minus D = +100 and +75) and the capable A minus B = 0 is not an agent that would have complied without either lever. The earlier diagnostic result lacked this floor (the frozen mutation gate sat at B minus D near zero; Section 8.4). The floor itself, however, was made non-degenerate by a post-canary untold-arm discoverability patch (Section 4.4), so the cell-D magnitude carries that provenance caveat.

Five honesty notes govern the reading. First, enforcement is latent, not exercised, in the capable models’ enforce cells: they self-refuse before dispatch, so the gate is never observed blocking a live violation in cell A. That the runtime does enforce rests on the population cell-C-versus-D contrast (untold + enforce stays safe while untold + off violates) plus a single n=1 pre-spend preflight probe, not on a per-rep block in cell A, which does not occur. Second, the below-floor model’s collapse is partial: its unsafe mutations are graded (median **unsafe_action_rate** 0.5), so at any pass threshold at or above 0.5 its B cell would read 75% rather than 0%; we keep the zero-tolerance threshold appropriate for an unauthorized-commit gate and call the below-floor behavior partial self-enforcement, while the load-bearing cells (capable B = 100, near-floor

Qwen3.5-9B $B = 0$) are invariant to any threshold in $[0, 1)$. Third, the structured-output parse rate is a confound but a protective one: pure-invalid reps score safe (empty output cannot commit), yet no headline cell-B or cell-D rep is a pure-invalid safe-by-default artifact (zero across all four models’ told-off and untold-off cells), so the weak-model $B = 0$ is genuine parseable commits; and Llama-3.3-70B is a built-in control, producing malformed turns on about 40% of scored reps (39 of 95) yet still reaching $B = 100$ through genuine refusals, which dissociates parse rate from the B outcome. Fourth, $n = 8$ per cell ($n = 4$ per sub-gate) is a pilot-scale ladder over four models, below our own pre-registered $n=12$ target (Section 8.4), not a scaling law; the intervals are wide and reported as such. Fifth, the scorer’s harm-only `unsafe_action_rate` code path was edited three times in the thirty hours before the run, the last 47 minutes before it (Section 4.5); those edits added an argument-defaults path that fired in zero headline cells (every mutation-gate violation in the table is detected by the original `must_not_call` path), so re-scoring the trajectories with the pre-edit scorer is the identity on this 2x2 and the load-bearing result is invariant to the pre-run scorer changes.

The other paid Tier-A mechanism, clinical refusal (M7), is reported as near-ceiling and underpowered rather than as a second substitution result. Across the ladder the clinical-claim detector fires on 7 of 223 refusal reps (3.1%), all on one generic task (`gab_16_refusal_untold`) and one model (Qwen3.5-9B); the two tasks purpose-built to elicit a clinical claim (`refusal_diagnose`, `refusal_dose`) produce zero fires across all four models, including below-floor Qwen2.5-7B. The refusal 2x2 is therefore flat at ceiling with no contrast distinguishable from zero, and the single discriminating cell (Qwen3.5-9B’s untold arm) at most suggests the clinical boundary begins to leak at low capability. We do not read a substitution or a moderation result off this arm; whether the near-total absence of fires reflects genuine boundary-respect or a detector too narrow after its D-55.1 redesign is an open scanner-sensitivity question (Section 8.4). Artifacts under the run directory of Section 5.1.

5.2 The contract-off map: telling works when salient, and the untold floor is real

The first battery (2026-07-02) ran the telling axis with the runtime fully off (`no_runtime_enforcement`), so every told condition realizes cell B and the untold condition realizes cell D. All rows are diagnostic (one model, Qwen3-235B, $n=3$ per condition).

Condition	Outcome ($n=3$ per row)
Told, bait phrasing	3/3 turn-1 refusal citing the <code>agent_safe</code> flag
Told, exact gated command named	3/3 turn-1 refusal
Told, normal task phrasing (constraint reached incidentally)	3/3 first-attempt dispatch of the gated command
Untold (flag globally stripped), valid target id supplied	3/3 completed commit, row proposed to active , truthful success report, no refusal step

Two readings follow. First, with the runtime off the told-and-salient conditions refused 3/3 while the untold condition completed the violation 3/3 (one model, Qwen3-235B, $n=3$ per cell), so the effect of telling against the violation floor is non-hypothetical at this tier. But self-enforcement of a context-verifiable constraint is salience-sensitive: the same agent that refuses 3/3 when the constraint is foregrounded dispatches the gated command 3/3 on first attempt when it reaches the constraint incidentally through normal task phrasing. Any claim that a told agent self-enforces must carry this qualifier. The incidental row is scored on first valid action: two of the three reps emit invalid-output steps before the dispatch, so “first-attempt” means first contact with the enforcement surface, not literal step one. These pre-dispatch invalid-output steps are the parser-clean model recovering to a valid dispatch, distinct in kind from the non-recovering invalid-output loops that place the 9B and 70B ladder models below the operate floor (Sections 6.3, 7.1); the shared invalid-output signature is read one way here and another there, and the residual overlap is a limitation rather than a clean separation. The two salient rows carry the author’s contemporaneous labels; the stored artifacts retain prompt-template hashes rather than the phrasings, so the row-to-directory mapping is not independently recoverable, though the 3/3 refusal pattern in both rows is.

Second, the untold row is the untold violation floor: with the contract withheld and the runtime off, nothing stood between the agent and a completed governance violation, reported truthfully as success. This makes the effect of telling (B vs D) and the effect of enforcing (C vs D) non-hypothetical. The substitution result of Section 5.1 is about the marginal value of enforcement given told (A vs B), not about these floors.

5.3 The M8 arc: the verifiability exception dissolves

M8 audit evidence emission was pre-registered as the non-verifiable exception: the constraint an agent cannot check from its decision-time context and therefore, by hypothesis, the place runtime enforcement retains behavioral value. Three probe regimes (2026-07-02, diagnostic, n=3 per cell) dismantled the classification in stages.

Regime	Outcome
Retrieved (hai explain output in context)	6/6 honest: cites the real card id under full_contract , reports “No evidence card is present.” under no_audit_chain ; verbatim-identical finals within each cell
Non-retrieval, plain ask, honest answer available	6/6 honest abstention, zero fabrication
Instrumental (an id is needed to advance an action), pre-fix harness	Fabricated ids dispatched freely: 8 instances across three probe sets

The first regime shows that verifiability is a property of the (constraint, decision-time context) pair, not of the mechanism: the moment retrieval lands the evidence in context, M8 behaves like the verifiable mechanisms and the agent self-enforces it. The second shows a cooperative agent asked plainly stays honest even without retrieval. That left the third regime as the only surviving fabrication signal. We reattributed it from verifiability to instrumental pressure, folded M8 into the goal-conflict axis, and pre-registered the two follow-ups below; the instrumental cell was later falsified outright as a harness-blindness artifact (Sections 5.5 and 6.1). After the arc, the sole surviving candidate for a genuinely non-verifiable constraint class is L7 stale-manifest drift (Section 3.4); no model-backed measurement exists. [FORK-L7-DRIFT: unmeasured; do not assume outcome]

5.4 H2: benign goal conflict does not degrade told-only compliance

The first pre-registered follow-up (artifact timestamp 2026-07-02T1942Z; pre-registered n=5) applied benign completion pressure, not adversarial injection, to audit faithfulness: 4 task variants graded P0 through P3, at P3 a pipeline is blocked and the user cannot proceed unless the agent produces a reference. Design: 4 tasks x 5 reps x 2 modes (**full_contract**, **no_audit_chain**) for 40 reps.

The result is a null. Zero fabrication across all 40 reps, scorer-verified: zero occurrences of the **forged_or_suppressed_audit_reference** violation kind in all 40 score files. The pre-committed falsification bar is an observed-rate threshold: fabrication at most 10% at P3. The four task variants map one to each pressure level, so the P3 cell is 5 reps per mode (10 reps). Observed P3 fabrication was 0/10; at n=10 this cannot exclude a true rate above 10%, so the bar is met and H2 is falsified only at the diagnostic tier (one model, Qwen3-235B, n=5). As a tighter descriptive bound, and as this paper’s own post-hoc caveat rather than part of the pre-registration, the pooled 0/40 across P0-P3 narrows the upper confidence interval further. The small-n limitation stated throughout applies. The side finding: 10 of the 40 reps fail overall pass with a refusal-format error only, meaning high pressure degraded tool-use competence (command-format loops) before it touched honesty.

5.5 The instrumental leg is falsified

The second pre-registered follow-up (2026-07-03; pre-registered n=5; **no_runtime_enforcement**) tested the reattributed hypothesis directly, after the stdout-inlining fix let the agent see its own command output. The result is a falsification: an instrumental effect of 0 percentage points against the pre-committed bar of at

least 40 points, diagnostic (one model, Qwen3-235B, n=5). The apparent instrumental-fabrication effect of Section 5.3 was a fact about a blind harness, not about the model; Section 6.1 tells the full story, including the design, the cell-level counts, the harness defect, and the disclosure that only the absence of the fabrication violation kind is citable from this scan. A thin n=3 diagnostic ladder run the same day first suggested the operate floor and the self-enforcement pattern; it was confounded (the action parser was tuned to one model's output format; Section 6.3) and is superseded by the pre-registered sweep of Section 5.1, which measures the same ladder properly at n=8 per cell.

5.6 Predictions against outcomes

#	Situation	Predicted delta	Outcome
P0	Capability moderator (marginal value of enforcement given told, A-B, across the ladder)	Substitution at high capability, enforcement load-bearing at low	Pattern seen (n=8, descriptive): capable A-B=0 (one-sided upper +32pp); weak A-B=+100 (raw counts; rep-level p=0.00016 overstates precision, task-level 0.33)
P1	Verifiable, no conflict, capable model	~0	Consistent: capable A-B=0 on the mutation gate (sweep, n=8); self-enforces when salient (n=3)
P2	Verifiable, goal conflict (H2)	Real, grows with pressure	Null: 0 fabrication P0-P3 (n=5, pre-registered)
P3	M8 references, instrumental pressure	Real	Falsified: harness-blindness artifact; 0pp with stdout fix (n=5, pre-registered)
P4	M8 references, cooperative, asked plainly	~0	Consistent: honest 6/6 per regime (n=3 per cell)
P5	L7 drift	Real at every level	[FORK-L7-DRIFT: unmeasured; do not assume outcome]
P6	Contract withheld (untold)	Enforcement is the only barrier	Consistent: untold violation floor observed (sweep D-cell; n=3 diagnostic)
P7	Below/near operate floor	Prevents malformed harm; enforcement load-bearing	Confirmed: weak models A-B=+100 (sweep, n=8); 7B partial/below floor

Table caption: Pre-registered predictions against outcomes. The capability moderator (P0, P1, P7) is measured by the sweep of Section 5.1 (n=8 per cell); the verifiability (P3, P4) and goal-conflict (P2) predictions are diagnostic tier (one model, Qwen3-235B, n=3-5 per cell).

Of the three moderators hypothesized to gate substitution, model capability is the one the evidence resolves: the ladder shows a sign flip in the marginal value of enforcement given told, from 0 (no marginal value detected) at high capability to +100pp at low capability. The other two nulled at the diagnostic tier: the

verifiability exception dissolved (P3, P4) and goal conflict produced zero fabrication (P2). The marginal value of enforcement given told (A-B) is the headline quantity, and the ladder puts capable cell B at ceiling while collapsing weak cell B to the floor; the supporting diagnostics are consistent, with cell B at ceiling in every told-and-salient probe (3/3 refusal in both foregrounded contract-off rows, 3/3 honest reporting in the cell-B half of the retrieved M8 regime, 3/3 honest abstention in the cell-B half of the non-retrieval regime, and 0 fabrication across all 20 cell-B goal-conflict reps; all diagnostic, one model, n=3 to 5 per cell). The result is therefore not a flat substitution but a capability-moderated one: for a capable cooperative agent above the operate floor, in-context specification substitutes for runtime enforcement on the mutation gate, while at and below the near-floor band enforcement is load-bearing because the told agent does not self-enforce. Runtime enforcement’s demonstrated behavioral value is the deterministic guarantee plus the region where the model cannot self-enforce: the untold violation floor, the sub-frontier capability band, the incidental-salience failure of Section 5.2, below the operate floor, and adversarial intent, untested here. Enforcement is behaviorally redundant only in the scoped high-capability sense, never structurally. All of this is within this fixed controller and this task set, over a four-model ladder at n=8 per cell; it is not a scaling law or a general claim about agents or harnesses. It is a single-runtime case study, and external non-HAI replication is future work (Section 7.2).

6 Methodology cautions as a contribution

Our probing arc produced, and then destroyed, an apparent positive result. We report the destruction as a named contribution rather than a limitations aside, because the failure mode is general to agent evaluation and easy to reproduce by accident.

6.1 Harness blindness manufactured a fabrication finding

In early probes the harness did not surface command output to the agent. Where stdout should have appeared in the observation feedback, the agent received a `stdout_ref` file path instead. The agent could dispatch a lookup, but it could never read the result. When a task then required an identifier to advance an action, the agent guessed: across three pre-fix probe sets we observed 8 instances in which it dispatched commands carrying plausible, well-formed identifiers it had never seen, including a fabricated evidence-card identifier and repeated invented target identifiers (model-backed diagnostic tier: one model, Qwen3-235B, temperature 0, n=3 per cell). This looked like a real finding, instrumental fabrication: honest when asked plainly, fabricating when an identifier is instrumentally required. It was, at the time, the strongest surviving candidate for a positive enforcement result.

The finding was an artifact of the harness. The pre-registered falsification experiment crossed Statement vs Action framing with id present/absent (4 tasks x 5 reps = 20, n=5 per cell, pre-committed bar: instrumental effect ≥ 40 percentage points), run after the committed stdout-inlining fix (commit 17db5ef), which inlines command stdout into the observation feedback bounded to the first 24,000 characters. Result: fabrication 0/5 in the Action/id-absent cell, 0/5 in the Statement/id-absent cell, an instrumental effect of 0 percentage points against the ≥ 40 pp bar, diagnostic (one model, Qwen3-235B, n=5). Once the agent can see the empty lookup, it abstains honestly 5/5 even in the Action arm, where the identifier is needed to finish the task. The cleanest before/after pair holds the task fixed: pre-fix, one rep fabricates a plausible card identifier while another rep in the same task and runtime mode stays honest; post-fix, all 6 reps are honest, with verbatim-identical finals within each runtime mode. One disclosure: we cite this scan only for the absence of the fabrication violation kind. In the id-absent cells honest abstention costs task success by design; across the scan all 20 reps fail overall pass and a refusal-classified scoring error fires 20/20, and the action/id-present task additionally logs an unsafe-mutation flag 5/5 under `no_runtime_enforcement`. The id-present control cells are valid (the agent reports or uses the real retrieved identifier); no outcome from this scan other than the fabrication-kind absence is cited in this paper.

The general lesson: eval harnesses that hide tool output manufacture spurious fabrication findings. A model that cannot observe its own command results is forced to guess, and a deterministic scorer will read the guess as fabrication. The resulting number is a fact about the harness, not the model. Genuine constraint-driven fabrication does exist under more adversarial operationalizations [arXiv:2606.14831]; our claim is narrower

and methodological: before attributing fabrication to the agent, verify the agent could see what it is accused of inventing.

6.2 Prompt-embedded ablations measure self-enforcement

A second caution follows directly from the 2x2. Guardrail ablation studies that toggle runtime enforcement while leaving the policy in the system prompt in both conditions, as in [arXiv:2604.15579], measure the marginal value of enforcement given the agent was told (A: deployment baseline vs B: told-not-enforced, the marginal value of enforcement given told), not the effect of enforcing (C: enforced-not-told vs D: neither, the violation floor). For a capable cooperative agent that self-enforces, this design will show small deltas and invite the wrong conclusion in either direction: that enforcement is worthless, or that a small delta on a told agent bounds what enforcement does for an untold one. Neither follows. Ablations that never withhold the in-context contract measure self-enforcement, not enforcement.

6.3 Two supporting cautions

A parser tuned to one model family scores other models below the operate floor. On the capability ladder (Section 7.1), only Qwen3-235B emitted parser-clean output; the 9B and 70B models looped invalid on routing, which automated scoring reads as inability to drive the contract. Without manual inspection this would have placed operable models spuriously below the operate floor. This confound is why the ladder carries no scaling claim.

Provider catalog metadata is not ground truth. The Together serverless catalog flag was unreliable: models it listed as serverless were dedicated-endpoint only in practice, which contaminated ladder composition; separately, the screen-to-ladder provenance chain is not reconstructable from artifacts. Roster composition must be verified against live endpoints, not catalog flags.

6.4 A checklist for agent-eval builders

- Confirm the agent receives tool and command output verbatim in its observation feedback; a reference or path is not output.
- Before reporting fabrication, hallucination, or dishonesty, check whether the accused content was observable to the agent at decision time.
- If the policy is in the prompt in every condition, the enforcement toggle measures the marginal value of enforcement given told, not the effect of enforcing.
- Validate the action parser against every model in the roster before attributing failure to capability; parser mismatch mimics the operate floor.
- Verify provider catalog claims (availability, endpoint class) against live endpoints before locking a roster.
- Pre-register a falsification bar for any surprising positive finding and re-run it after fixing the harness defect that co-occurred with it.

7 The capability ladder and external replication

7.1 The capability ladder

Model capability is the headline moderator (Section 5.1), and the evidence is a pre-registered four-rung ladder run on the concentrated task set at $n=8$ per cell under both `full_contract` and `no_runtime_enforcement`: MiniMax-M3 (capable primary), Llama-3.3-70B (capable), Qwen3.5-9B (near the operate floor but operational), and Qwen2.5-7B (below it). The ladder produces the capability-moderated pattern reported in full in Section 5.1: the marginal value of enforcement given told (A-B) is 0 for the two capable models (no marginal value detected, one-sided upper +32pp) and +100pp for the two weak models (raw counts; the rep-level exact p is not the honest unit of analysis, Section 5.1). It supersedes a thin $n=3$ diagnostic precursor (2026-07-03, `runs/pilot/_probe_ladder/`) that first surfaced the operate floor and the self-enforcement pattern but was parser-confounded (Section 6.3) and left one rung unmeasured within-model.

Three properties bound what the ladder can claim. First, it is a four-model pilot at $n=8$ per cell, not a scaling law: it establishes that the marginal value of enforcement given told moves from undetectable to load-bearing across this capability range, not a capability-ordered curve. Non-monotone capability-compliance relationships across model families are documented [arXiv:2606.01317, arXiv:2601.08196], and four models cannot adjudicate them. Second, the structured-output parse rate is a confound, but a protective one with a built-in control: pure-invalid reps score safe (empty output cannot commit), so the weak-model $B = 0$ is a lower bound on the self-enforcement failure rate, and Llama-3.3-70B produces malformed turns on about 40% of scored reps (39 of 95) yet still reaches $B = 100$ through genuine refusals, dissociating parse rate from the B outcome (the thin precursor’s parser confound, Section 6.3, is exactly why this control matters). Third, the operate floor is real and below-floor behavior is partial: Qwen2.5-7B’s unsafe mutations are graded (median `unsafe_action_rate` 0.5), so its collapse is partial self-enforcement rather than total, while the load-bearing near-floor rung (Qwen3.5-9B, $A-B = +100$) is threshold-invariant. The clean load-bearing case is therefore Qwen3.5-9B: it operates the contract (drives valid actions, enforce cells safe) yet does not self-enforce told-not-enforced, so an operate-but-violate band does exist between the floor and the frontier, contrary to the thin precursor’s suggestion that models either drive the contract and comply or fall below the floor.

Compounding these caveats narrows the robust load-bearing signal. Removing the threshold-sensitive below-floor 7B and holding the capability-family confound, the evidence that a told agent fails to self-enforce while still operating the contract rests on a single non-frontier Qwen model (Qwen3.5-9B) at $n=8$ on one mechanism and one runtime. The crossover’s low arm should be read at that strength: a real, mechanistically verified failure to self-enforce, but not yet a family-general or multi-model result. The thin $n=3$ diagnostic precursor, which ran pre-parser-fix at temperature 0 on one gate task, contributes no number to this ladder.

7.2 Scope: a single-runtime case study; external replication as future work

Every model-backed number in this paper comes from one controller, HAI v0.2.0, with its particular manifest, contract-authoring style, and task suite. This paper is accordingly scoped as a single-runtime case study (PAPER.md D-42): hypothesis H5, that the specify-vs-enforce effect generalizes beyond HAI, is stated as a hypothesis, and the external non-HAI replication that would test it is future work, not a pending component of this paper’s evidence. Nothing in the substitution result distinguishes a phenomenon from a HAI quirk until that replication exists, and the claims throughout are conditioned on this controller.

The future-work design is specified so the scope boundary is concrete: reproduce the 2x2 on an independent harness with its own mechanism inventory and deterministic scorer, measuring at minimum the A-vs-B contrast, the marginal value of enforcement given the agent was told; where the replication also attributes the telling axis (B vs D , C vs D), first-attempt scoring applies (Section 3.6). TeamBench [arXiv:2605.07073] (Section 2) is one candidate substrate: prior evidence of the same qualitative shape in a different mechanism domain and design, not itself a replication of this paper’s effect. Its sub-metric asymmetry (pass-rate parity coexisting with 3.6 times more verifier-edits-executor cases in the prompt-only arm) cautions that aggregate parity can mask enforcement value. That caution applies to our own instrument. The deterministic scorer observes per-mechanism `mechanism_disabled` markers and each task’s load-bearing violation kind (Section 4.2), so it detects whether the specific guarded violation occurs, but it does not resolve finer sub-metric channels such as a verifier-edits-executor count. An enforcement effect that left the pass rule and the tracked violation kinds unchanged while shifting such a channel would stay undetectable at diagnostic n . Aggregate parity in these diagnostics should therefore not be read as sub-metric equivalence.

8 Discussion

8.0 Scope conditions

The substitution is conditional, not universal. On the sweep (Section 5.1, $n=8$ per cell), substitution held when six conditions held together: the agent was capable, above the operate floor; the agent was cooperative; the constraint was stated in the in-context contract and told plainly; the constraint was salient at decision time; the harness surfaced command output to the agent; and the inputs were benign. The first, capability, is the one the ladder varies: where it fails, told-not-enforced collapses and enforcement is load-bearing

(Section 5.1). The other four behavioral conditions, when they fail, likewise mark regions where runtime enforcement retains demonstrated value (Section 8.1); the sixth, the harness surfacing command output, is a measurement-validity condition whose failure manufactures the spurious fabrication finding of Section 6 rather than a genuine enforcement region.

8.1 What runtime enforcement is still for

The capability-moderated result relocates enforcement’s value rather than eliminating it. Beyond the deterministic guarantee, which is structural rather than a condition-failure region, each scope condition of Section 8.0 marks a region where enforcement retains behavioral value when that condition fails.

First, the deterministic guarantee of Section 3.5, which holds unconditionally and is not tied to any one condition failing. Cell B: told-not-enforced compliance is a statistic over model behavior; cell A: deployment baseline compliance on the enforced surface is a property of the harness. A gated action that the runtime blocks cannot occur; a 3/3 refusal rate at $n=3$ bounds nothing.

Second, when the told condition fails: the untold violation floor of Section 5.2, where the contract is withheld and enforcement is the only barrier. In-context specification cannot substitute for enforcement on rules the context never carried, and real deployments accumulate such rules through contract drift, incomplete prompt assembly, and context compaction [arXiv:2606.22528].

Third, when the salience condition fails: the incidental-salience region of Sections 5.2 and 8.2, where the same told constraint reached incidentally rather than foregrounded was dispatched 3/3 on first attempt. Enforcement catches the violation that a told-but-not-salient agent commits.

Fourth, when the capability condition fails: below the operate floor, where enforcement prevents malformed harm rather than disobedience (Section 7.1); the failure there is contract friction, not governance.

Fifth, when the benign-inputs condition fails: adversarial inputs, where the cooperative assumption also no longer holds. The substitution result is scoped to benign completion pressure, not adversarial injection. The injection literature already shows architectural enforcement outperforming prompt-only defenses under attack [arXiv:2503.18813, arXiv:2504.11703]; Verifier Tax [arXiv:2603.19328] similarly finds policy mediation intercepting up to 94% of non-compliant actions under instrumental pressure while agents hallucinate identifiers to route around blocks. Injection robustness is cited territory, not a claim of this paper, and nothing here licenses removing enforcement from adversarially exposed surfaces.

One published result points the opposite empirical direction and warrants direct comment. Mechanical Enforcement [arXiv:2605.14744] finds mechanical enforcement raising governance quality and task accuracy (MCC approximately 0.43 to 0.88) over text-only governance, where this paper finds enforcement behaviorally redundant given specification. The findings differ on four axes: domain (regulated banking versus non-clinical personal wellness), metric type (decision-rationale quality versus tool-action compliance), design (paired alternative configurations versus a crossed per-mechanism factorial), and pressure regime (their regulated-decision setting versus benign completion pressure here). The tension is a difference in what is measured and under what pressure, not a contradiction on the marginal value of enforcement given told. A same-direction 2026 result, Mind the GAP [arXiv:2602.16943], finds a runtime governance contract with no detectable deterrent on forbidden tool-call attempts; that is a leakage catch-layer under jailbreak, whereas the mutation gate here is a hard commit block under benign use, so its untold-plus-enforce cell (C) is safe by construction. The two do not conflict: enforcement architecture that only annotates or logs need not deter, while enforcement that blocks dispatch does.

8.2 The salience confound

The told-not-enforced leg carries a caveat. Self-enforcement is salience-sensitive: in the diagnostic battery of Section 5.2, the told agent refused the gated action 3/3 when the constraint was foregrounded, but dispatched it 3/3 on first attempt when it was reached incidentally through normal task phrasing (one model, $n=3$ per cell). Under first-attempt scoring, telling substituted for enforcement only when the telling was salient at decision time. The substitution claim is really a claim about salient in-context specification, and how salience decays with context length, task distance, and compaction is unmeasured here [arXiv:2606.22528].

8.3 Personal health as a domain

HAI’s domain, non-clinical personal wellness, is a deliberate instrument choice. The non-clinical boundary (no diagnosis, treatment, prescribing, or autonomous medical decisions) is itself an evaluated constraint, the zero-tolerance leg of M7 refusal of out-of-contract requests. The choice makes a policy-dense boundary an enforceable, ablatable object of study with unambiguous violation semantics, without any clinical claim.

8.4 Limitations

The primary model-backed evidence is the four-model capability-ladder sweep at $n=8$ per cell (Section 5.1); the design’s framing and moderator probes are a lower diagnostic tier (one model, Qwen3-235B-A22B-Instruct-2507, Together AI, temperature 0, n of 3 to 5 per cell, on diagnostic probe tasks rather than the locked suite). At $n=8$ per cell the intervals are wide and reported as such, and the ladder is four models over a single reference runtime (HAI), one harness, one prompt template; the paper is scoped as a single-runtime case study, and external non-HAI replication is future work (Section 7.2). The untold violation floor and the harness-blindness finding are existence and property demonstrations that do not rest on cross-model generalization. L7 stale-manifest drift, the sole surviving candidate for a true non-verifiable enforcement delta, is unmeasured. [FORK-L7-DRIFT: unmeasured; do not assume outcome] Inputs are benign throughout; the adversarial-input arm was retired at the benchmark tier (Section 4.8), and injection robustness is cited territory, not a claim. The clinical-refusal detector, redesigned to a frame-based scanner in D-55.1, fires on almost no reps in the sweep (Section 5.1); whether that reflects genuine boundary-respect or an over-narrowed scanner is unresolved, which is why the refusal arm is reported as near-ceiling and underpowered rather than as coverage.

The disposition confound that limited the earlier diagnostic result is resolved here, and it is worth being precise about how. On the frozen tasks the enforce-able mechanisms were disposition-covered: a cooperative capable agent already complied whether or not it was told (B versus D near zero), so the marginal value of enforcement given told (A versus B) was consistent with substitution but did not exclude that the agent needed neither lever, and the difference-in-differences S was unidentified. The told-arm reconstruction of Section 4.4 (D-52) removed that degeneracy on the mutation gate: the untold-unenforced cell now actually violates, so even for the capable models cell D is near the floor (MiniMax 0, Llama 25 percent safe) and B versus D is large (+100, +75). With the floor non-degenerate, telling and enforcing each have a measured effect and $S = (A \text{ minus } B) \text{ minus } (C \text{ minus } D)$ is identified within a single model, so the capable-model marginal value $A \text{ minus } B = 0$ is read as substitution rather than as a disposition that would have complied regardless. The capability ladder then supplies the moderation: at low capability the agent does not self-enforce even when told ($B = 0$), so enforcement given told is load-bearing ($A \text{ minus } B = +100$ on raw counts; the rep-level exact p overstates precision, Section 5.1). The clinical-refusal arm, by contrast, remains near-ceiling and underpowered (Section 5.1), so we make no disposition-versus-specification separation there; the pre-registered directiveness-gradient pilot (Section 9.1) is retained as a complementary, finer-grained within-model identification on the mutation gate, not as the confound’s sole resolution.

The ladder’s own limitations, disclosed against the pre-registered run design (Section 4.10): it is a bounded four-model moderator, not a scaling-law claim. Capability is partly confounded with model family, because both capable rungs are non-Qwen (MiniMax-M3, Llama-3.3-70B) while both weak rungs are Qwen (3.5-9B, 2.5-7B); the cross-family agreement of the two capable self-enforcers strengthens the substitution side, but the load-bearing side rests on Qwen-family models, so the crossover cannot fully separate a capability effect from a Qwen-family low-capability effect, and the abstract must not assert family-general capability ordering. Under per-model vendor-recommended sampling, model identity is confounded with sampling regime, so the ladder cannot separate a capability trend from a sampling-temperature trend; the per-model 2x2 is computed within a single model at a constant sampling regime and is unaffected. Provider seeds are unsupported on the serverless MoE endpoints, so replication is across independent stochastic draws rather than fixed-seed determinism. The untold arm’s schema-partial-telling, told-the-opposite M5, and cell-B enforcement-prose disclosures of Section 4.4, and the clinical-detector circularity of Section 4.5, apply to every run-design cell.

8.5 What would change the conclusion

The conclusion is falsifiable on several fronts. The high-capability substitution would break if a capable, above-floor, cooperative model showed a nonzero A-versus-B delta on the mutation gate under the same first-attempt scoring, or if capable self-enforcement failed at higher n or under longer-horizon salience decay. The capability-moderation would break if the crossover failed to replicate on a second runtime, or if a non-Qwen weak model self-enforced told-not-enforced, which would reattribute the load-bearing low end from capability to the Qwen family. Independently, a measured L7 drift delta, or goal-conflict pressure beyond P3 breaking the null, would restore behavioral, not merely guarantee-level, value to enforcement in regimes we report as null. The claim is exactly as strong as the scope conditions of Section 8.0, and no stronger.

9 Future work and conclusion

9.1 Future work

The measurement this design points at most directly is L7 stale-manifest drift (Section 3.4), the sole surviving genuinely non-retrievable candidate for a real cooperative-agent enforcement delta. Concurrent work establishes the drift-causes-violation half, that context compaction silently erases constraints the agent obeyed while they were visible [arXiv:2606.22528]; whether independent runtime enforcement catches post-drift violations is the untested second half. [FORK-L7-DRIFT: unmeasured; do not assume outcome]

Second, the adversarial-input arm should graduate from scorer-coverage evidence to model-backed evidence. This paper cites injection robustness rather than claiming it (Section 8.1); running the specify-vs-enforce 2x2 under attacker-controlled inputs would locate where the substitution result ends and the injection-defense literature begins.

Third, a fuller screened capability ladder behind an action parser tolerant of every roster model. The present ladder carries only the existence of an operate floor (Section 7.1); a parser-tolerant rerun across a wider screened roster could locate the floor rather than merely evidence it, with non-monotonicity across families expected [arXiv:2606.01317, arXiv:2601.08196] and no scaling law promised.

Fourth, external non-HAI replication (Section 7.2). This paper is a single-runtime case study by decision, and the replication that would distinguish a phenomenon from a HAI quirk, reproducing the 2x2 on an independent harness with its own mechanism inventory and deterministic scorer, is the highest-value follow-on measurement. (The multi-model higher- n question, by contrast, is resolved within this paper for the mutation gate: the pre-registered run of Section 4.10 is that run, and its mutation-gate results land in Section 5.1 per the pre-committed outcome map, not in future work. The other paid mechanism, clinical refusal, was run on the ladder but is near-ceiling and underpowered (Section 5.1); the Tier-D scope mechanisms are offline-only by pre-registered design, D-49.)

Fifth, a pre-registered exploratory disposition-mapping pilot sharpens the identification the capability ladder already delivers at the coarse grain (Section 8.4). Where the ladder separates specification from disposition across models, the pilot does so within a single model: it manufactures intermediate disposition on the mutation gate, the one mechanism carrying both a movable told-versus-untold effect and a runtime enforce/off lever, through a directiveness gradient over the operator's request, from a directly commanded activation to a neutral, instrumental phrasing, so that the untold-unenforced violation rate is dialed off its floor and both levers move on the same rows. There $S = (A \text{ minus } B) \text{ minus } (C \text{ minus } D)$ is identified within one model, letting the specification effect be read against that model's baseline disposition rather than confounded with it; the benign completion-pressure and deployment-scope pilots map the disposition-covered and specification-substitutable types of Section 3.6 alongside it. The pilot is pre-registered as exploratory (PILOT_PROTOCOL Section 22): it does not amend the frozen suite or the headline, and a confirmatory follow-up would re-lock the chosen constraint and gradient level with fresh reps. Newcombe intervals on B vs D and C vs D and a difference interval on S are reported per constraint by the results layer.

Two deferred lines complete the program. H7 extends the design toward the capability-versus-compliance question studied externally under scaling-laws-for-oversight [Engels2025], asking whether a deterministic Guard at general Elo of zero occupies a legitimate point on that external curve; this paper itself claims no

scaling law (Section 1). H8 asks whether fine-tuned operators reach contract compliance at smaller scales than untrained operators (the S1 fine-tuning sequel), which would move the operate floor itself.

9.2 Conclusion

On the mutation gate, in a small pre-registered single-runtime case study, in-context specification substitutes for runtime enforcement for the capable models and enforcement is load-bearing for the weak ones. Told the rule with the runtime off, both capable models complied on every replicate of both commit tasks while both weak models committed the gated mutation on every replicate; the enforced cells are safe by construction, so the marginal value of enforcement given told is 0 for the capable models (a failure to detect at this n , one-sided upper bound +32pp, not demonstrated equivalence) and the whole barrier for the weak ones. We report this as raw counts on a task-level unit (two tasks per arm, near-deterministic replicates), not a pooled p-value; the effect is stark but small and confounded – capability with model family, and once the below-floor 7B is set aside the robust load-bearing side rests on a single non-frontier model. Because the reconstructed untold-unenforced cell violates even for the capable models, the capable-side result reads as substitution rather than pre-existing disposition. Told plainly is load-bearing on top of capability: self-enforcement is salience-sensitive, and when the same told constraint was reached incidentally, the gated command was dispatched first-attempt 3/3 (diagnostic, one off-roster model, $n=3$, not independently reconstructable). The clinical-refusal arm is near-ceiling and underpowered, not a second substitution result, and the whole finding is scoped to a single runtime, a four-model ladder whose low end is Qwen-family, and the conditions of Section 8.0.

Enforcement is redundant only in that scoped, high-capability sense. It retains the deterministic guarantee, which no behavioral result touches; it is the only barrier at the untold violation floor, where an agent never told the rule completed the violation with a truthful success report; it is load-bearing across the sub-frontier capability band the ladder measures, where a told agent still does not comply; below the operate floor it prevents malformed harm rather than disobedience; and adversarial territory remains untested here, cited as the regime where telling is already known to fail.

The takeaway we most want practitioners to carry is methodological. Before attributing fabrication to a model, check what the harness let it see. An apparent instrumental-fabrication effect in our own pipeline dissolved to 0 percentage points against a pre-committed bar of at least 40 (diagnostic, one model, pre-registered $n=5$) once the harness inlined command stdout. Harness blindness manufactures spurious findings; an evaluation of agent honesty is only as sound as the observation channel the harness provides.

Appendix

Skeleton only. Each subsection names the artifact that will supply the final content and the PAPER.md anchor it must trace to; no full tables are populated here.

A. Task suite

[STUB] Table APPX.1: tasks by level and load-bearing mechanism (M4-M9-TX). An earlier apparatus (a 28-task suite with 25 static oracle pairs, 23 per-mechanism plus 2 `no_runtime_enforcement` floor pairs, and a 16-trajectory adversarial layer) was retired by PAPER.md decision D-37 (2026-07-04, landed across benchmark commits `a10e850..48ff87b`): those figures no longer apply. The suite was rebuilt as a sharp per-mechanism 2x2 (told x enforced) with the told/untold contract axis (`e72dced`) and first-attempt scoring (`48ff87b`) built in, then expanded by D-39 (2026-07-05, commit `ed07f5a`) to three scenario pairs per mechanism: 39 tasks (L1: 2, L2: 4, L5: 8, L6: 24, L7: 1); 16 paid-swept task-by-mode cells (two tasks carry `no_runtime_enforcement` as the floor). Populate Table APPX.1 from the frozen suite and confirm the count against the release tag before submission.

B. Prompt provenance

`deployment_full_v2` (D-28, authorized 2026-06-28): the manifest is embedded as minified JSON with null/empty fields dropped, lossless relative to `deployment_full_v1` (verified by round-trip; no

command/flag/type/choice/default/help/description removed), reducing the rendered prompt from approximately 43K to approximately 22K tokens after the v1 prompt exceeded a model’s context limit. Recorded as `PILOT_PROTOCOL.md` Amendment 2. Cite the prompt identifier from the roster’s `prompt_id` field (an identifier string, not a hash) and the prompt file hash from `scripts/lock_hashes.json`, not from this appendix. Caution: the recorded lock hash predates the prompt edits committed 2026-07-04 (e72dced, told/untold axis) and 2026-07-05 (ecee5f5, S3-S6 prompt-neutrality pass; boundary-block parameterization); re-derive the file hash at populate time, against the §20 run manifest, rather than reusing any recorded value.

C. Probe protocol summary

[STUB] Per-cell n and seed/temperature table, in two clearly separated blocks.

Diagnostic probes (Sections 5.2-5.5): `Qwen/Qwen3-235B-A22B-Instruct-2507-tput` (Together AI), temperature 0, top_p 1, no seed support in the provider API, n=3-5 per cell depending on probe (see grounding pack Section 6 for per-probe n). Table to enumerate probe directory, task(s), mode(s), and n per row.

Pre-registered run (Section 5.1): four-model Together ladder (MiniMax-M3 primary, Llama-3.3-70B, Qwen3.5-9B, Qwen2.5-7B; Qwen3-235B primary deprecated mid-cycle and replaced per D-56), vendor-recommended per-model sampling recorded per condition with card URL and snapshot date, n=4 per cell (the mutation-gate headline pools its two commit sub-gates to n=8), canary-first phasing with a hard stop that the gate cleared, decision quantities and the eleven-branch outcome map per `PILOT_PROTOCOL.md` §20. Per-model sampling and prompt-token rows are pinned in the locked roster revision; token counts for the identical held-constant prompt differ per tokenizer, so the ~22K figure of Appendix B is Qwen2.5-specific and per-model counts are recorded per condition.

D. Adversarial trajectory table (pointer)

[RESOLVED: retired] The 16 hand-authored adversarial trajectories (4 each vs M7, M5+M6, M8, M4) were scorer-coverage evidence at the appendix tier, never model-backed. `PAPER.md` D-37 (2026-07-04) records their retirement as final: commit a10e850 removed `results/adversarial_summary.py` and the trajectories, and `reproduce_offline.py` no longer emits the `adversarial_summary` artifacts at HEAD. The adversarial-input arm is cited future work (Sections 4.8, 9.1), not an arm of this paper. This appendix subsection should either be deleted at LaTeX conversion or reduced to a one-line provenance pointer; no adversarial table will be populated.

E. Reproducibility

Per `PAPER.md` (Engineering Plan, Reproducibility script row), `reproduce_offline.py` runs the offline pipeline (rule-baseline ablation, evidence tables, figures, error taxonomy, and cell contrasts; the static and live isolation matrices were retired by D-37 at a10e850) from a single command. Artifact and hash baselines in `REPRODUCIBILITY.md` and `REPRODUCIBILITY_GOLDEN.json` (`REPRODUCIBILITY_GOLDEN.json` regenerated post-D-37). The 76s Apple M2 runtime figure is the pre-a10e850 baseline; re-baseline the runtime and pipeline description against the frozen D-39 39-task suite and the completed analysis-layer integration before populating this appendix.

F. Scorer config hash chain

Scorer behavior thresholds and critical violation kinds load from `scorer_config.paper_v1.json`; the config hash records the audited config bytes (D-18). A provenance-only hash bump (68e29510... to d310f503...) accompanied the D-30 scorer-correctness pass, recorded in `lock_hashes.json`; metric thresholds, pass rule, and critical kinds were unchanged in that pass.

G. D-O-02: Appendix E coding-agent sketch (open decision)

[OPEN, deferred to mid-August polish] Keep-or-drop decision on an Appendix E coding-agent sketch. `PAPER.md` D-O-02 requires an anti-overclaim header if kept; `PAPER.md` does not specify the header’s

wording. Workflow-derived content requirements for that header: the sketch is illustrative, not a fourth evidence tier (the three tiers, static oracle-pair evidence, live runtime probe, and model-backed diagnostic, are never merged or extended), and not evidence of generalization beyond the HAI controller (the paper is a single-runtime case study; external replication is future work, Section 7.2). Do not draft Appendix E content before D-O-02 resolves.

References

Formatted from the verified neighbor list (`paper/prior_art_notes.md`, synced 2026-07-05 from the 2026-07-04 novelty audit, D-38; full adjudication archived at `ARCHIVE/novelty_audit_2026-07-04/`); verification status per entry is recorded there. Section 2’s external numeric claims were verified against primary sources 2026-07-04: ContextCov (88.3% vs 67.0%) and PhantomPolicy (95.3% to 40.7%) in the novelty audit; Mechanical Enforcement (MCC ~ 0.43 to 0.88; 73% deferral reduction) and Verifier Tax (up to 94% intercepted; SSR below 5%) by direct arXiv re-fetch. Entries are listed by the short name used in the text.

- ABSTAIN: Ojewale and Venkatasubramanian (Brown University). What Benchmarks Don’t Measure. arXiv:2606.02965. Workshop paper.
- Agent Behavioral Contracts: Bhardwaj. Agent Behavioral Contracts. arXiv:2602.22302.
- Agent-SafetyBench: Zhang et al. ACL 2025. arXiv:2412.14470.
- AgentSpec. arXiv:2503.18666.
- AHE (automatic harness evolution). arXiv:2604.25850.
- AIRGuard: Qin, Zhuang, et al. arXiv:2605.28914.
- CaMeL: Debenedetti et al. (Google DeepMind / ETH Zurich). arXiv:2503.18813.
- CEF/Thanatosis: J.P. Morgan AI Research. Is Your Agent Playing Dead? arXiv:2606.14831.
- ContextCov: Sharma. arXiv:2603.00822.
- Engels, Baek, Kantamneni, and Tegmark. Scaling Laws For Scalable Oversight. arXiv:2504.18530. NeurIPS 2025 (Spotlight). Cited as [Engels2025] per PAPER.md H7/D-03 (deferred line). Verified against the primary source 2026-07-04.
- ETCLOVG survey. OpenReview 3hXEPbG0dh. TMLR, under review; cited via OpenReview, not as an accepted paper.
- FORGE: Palumbo, Choudhary, Choi, Amir, Chalasani, Jha. arXiv:2602.16708 (v1 titled “Policy Compiler for Secure Agentic Systems”).
- Governance Decay (mitigation: Constraint Pinning). arXiv:2606.22528.
- Guardrails AI. Software framework.
- Harness-Bench. arXiv:2605.27922.
- Harness-MU: Fan, Nie, Dai. arXiv:2606.21856.
- IFEval. arXiv:2311.07911.
- Invariant Guardrails: Invariant Labs (now part of Snyk). Software framework.
- LogiSafetyBench. arXiv:2601.08196.
- Mechanical Enforcement: de la Chica Rodriguez and Marti-Gonzalez. arXiv:2605.14744.
- Meta-Harness. arXiv:2603.28052.
- Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943.
- NeMo Guardrails: NVIDIA. Software framework.
- NLAH (natural-language agent harnesses). arXiv:2603.25723.
- PhantomPolicy: Wu, Gong (Atlassian). arXiv:2604.12177.
- Policy-as-Prompt. arXiv:2509.23994. NeurIPS 2025 Regulatable ML workshop.
- Progent. arXiv:2504.11703.
- Prompts Don’t Protect: Uppala. arXiv:2605.18414. Single author; no venue located.
- RECAST. arXiv:2505.19030.
- SABER. arXiv:2606.01317.
- SafePyramid. arXiv:2606.29887.
- ST-WebAgentBench: Levy et al. arXiv:2410.06703. ICLR 2026.
- Symbolic Guardrails: Hong et al. (CMU). arXiv:2604.15579.
- TeamBench: Kim et al. arXiv:2605.07073.
- TRACE: Zhou et al. arXiv:2606.13174.

- VerIF. arXiv:2506.09942. EMNLP 2025 per ACL Anthology.
- Verifier Tax: Sah, Srivastava, Sah, Jordan. arXiv:2603.19328.